TRANSPOSITION CIPHERS

A **transposition cipher** does not substitute one symbol for another. Instead, it changes the location of the symbols. A symbol in one position of the plaintext may appear in a different position in the ciphertext. In other words, a transposition cipher **reorders (transposes) the symbols**.

A transposition cipher reorders symbols.

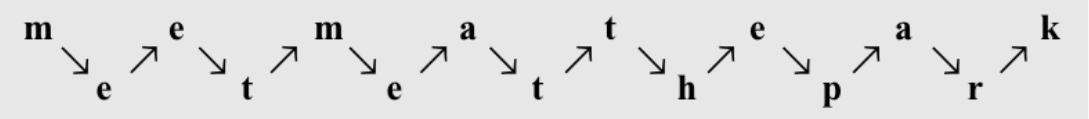
Keyless Transposition Ciphers

Simple transposition ciphers are keyless. The textbook describes two methods for permuting characters:

- In the first method, the text is written into a table **column by column** and then transmitted **row by row**.
- In the second method, the text is written into the table **row by row** and then transmitted **column by column**.

Example 3.22 — Rail Fence Cipher

The **rail fence cipher** is an example of the first method. The plaintext is arranged in two lines as a zigzag pattern, and the ciphertext is created by reading the first row followed by the second row. To send “Meet me at the park,” Alice arranges the letters as shown below.



The ciphertext is **MEMATEAKETETHPR**. Bob divides the ciphertext in half: the first half forms the first row and the second half forms the second row. He then reads the letters in zigzag order to recover the plaintext. In this example, the second half has one character less than the first half. Since there is no key and the number of rows is fixed at two, the textbook notes that this cipher is easy to analyse if the method is known.

Example 3.23 — Row-Wise Writing and Column-Wise Reading

Alice and Bob can agree on the number of columns. In this example, the same plaintext is written row by row in a table with four columns:

m	**e**	**e**	**t**
m	**e**	**a**	**t**
t	**h**	**e**	**p**
a	**r**	**k**	

Reading the characters **column by column** gives the ciphertext **MMTAEEHREAEKTTP**. For decryption, Bob writes the received message column by column and reads the table row by row. If Eve knows the number of columns, she can easily decipher the message.

Position Permutation

Example 3.24 shows how each character position in the plaintext is moved to a position in the ciphertext. For the four-column arrangement, the positions follow a regular pattern with a difference of four between adjacent positions in each group. This regularity can help an attacker analyse the cipher.

01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
01	05	09	13	02	06	10	13	03	07	11	15	04	08	12

Source note: *The printed position map and its accompanying list appear to contain a typographical error in one entry. From the four-column arrangement, the corresponding position is 14, which maintains the stated difference of four.*

Keyed Transposition Ciphers

A keyed transposition cipher divides the plaintext into groups of a predetermined size, called **blocks**, and uses a **permutation key** to rearrange the characters in each block separately. Unlike a keyless method that reorders the whole plaintext using the writing and reading arrangement, keyed transposition applies the key block by block.

Example 3.25 — Block Permutation

Alice needs to send the message “Enemy attacks tonight” to Bob. They divide the text into groups of five characters. A filler (bogus) character **z** is added at the end so that the last group has the same size as the others. After removing spaces and adding the filler, the text is grouped as follows:

```
ENEMY | ATTAC | KSTON | IGHTZ
```

The encryption key is **(3 1 4 5 2)**. It means that the third character of each plaintext block becomes the first character of the ciphertext block; the first becomes the second, and the remaining characters follow the order given by the key.

Encryption key	3	1	4	5	2
Position	1	2	3	4	5

The blocks are permuted as shown:

```
ENEMY → EEMYN
ATTAC → TAACT
KSTON → TKONS
IGHTZ → HITZG
```

Alice sends the ciphertext **EEMYNTAACTTKONSHITZG**. Bob divides it into five-character groups and uses the inverse key **(2 5 1 3 4)** to recover the plaintext, then removes the filler character.

Combining Two Approaches

More recent transposition ciphers combine the two approaches to achieve better scrambling. Encryption and decryption use three steps:

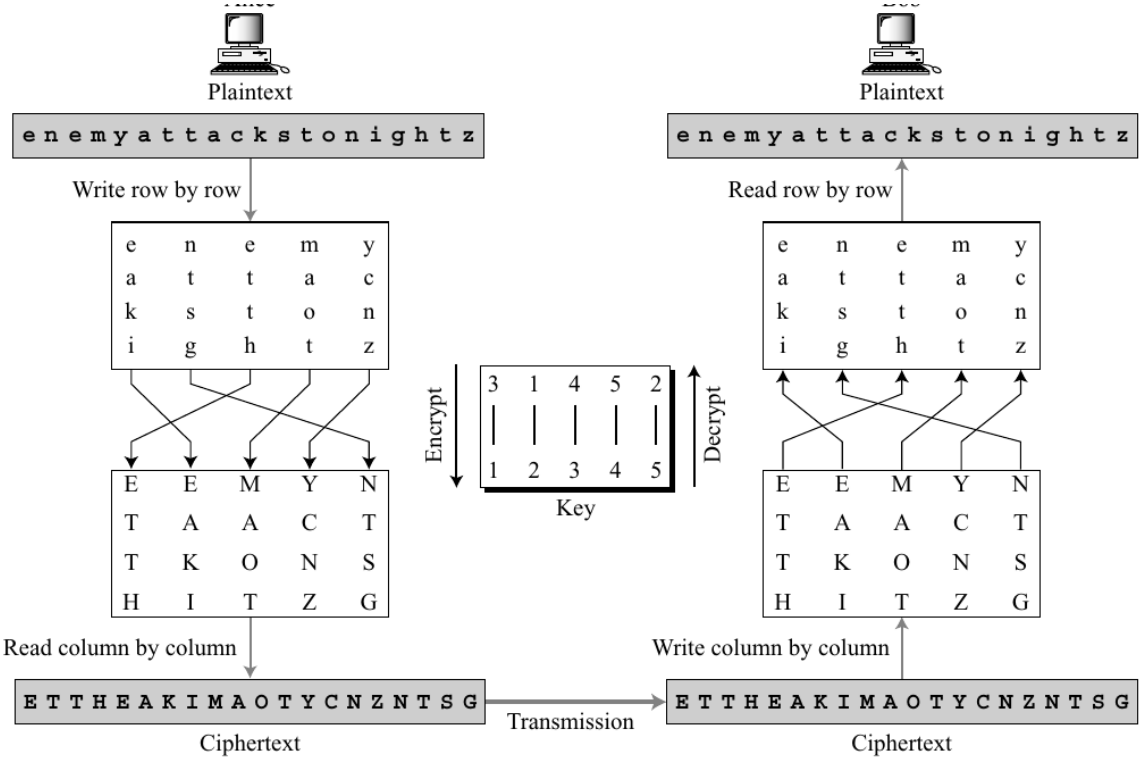
1. Write the plaintext into a table row by row.
2. Permute the columns by reordering them according to the key.
3. Read the new table column by column to form the ciphertext.

The first and third steps provide keyless global reordering; the second step provides blockwise keyed reordering. These ciphers are called **keyed columnar transposition ciphers**, or simply **columnar transposition ciphers**.

Example 3.26 — Combined Method

The same message used in Example 3.25, **enemyattackstonightz**, is encrypted with the combined approach. The plaintext is written row by row in five columns. The columns are permuted using the key **(3 1 4 5 2)**, and the new table is read column by column. The resulting ciphertext is:

```
ETTHEAKIMAOTYCNZNTSG
```

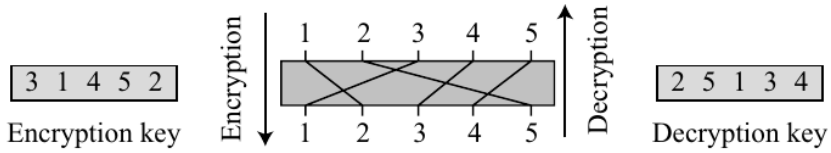


Textbook Figure 3.21: Combined columnar transposition and decryption process.

For decryption, Bob writes the ciphertext column by column into a table, restores the columns using the decryption key, and reads the restored table row by row. The operations are performed in the reverse order of encryption.

Keys in Columnar Transposition

A graphical column permutation can be represented by two keys: an **encryption key** and a **decryption key**. Each key is stored as a list with one entry for each column. An entry shows the source column number; the position of that entry gives the destination column number.



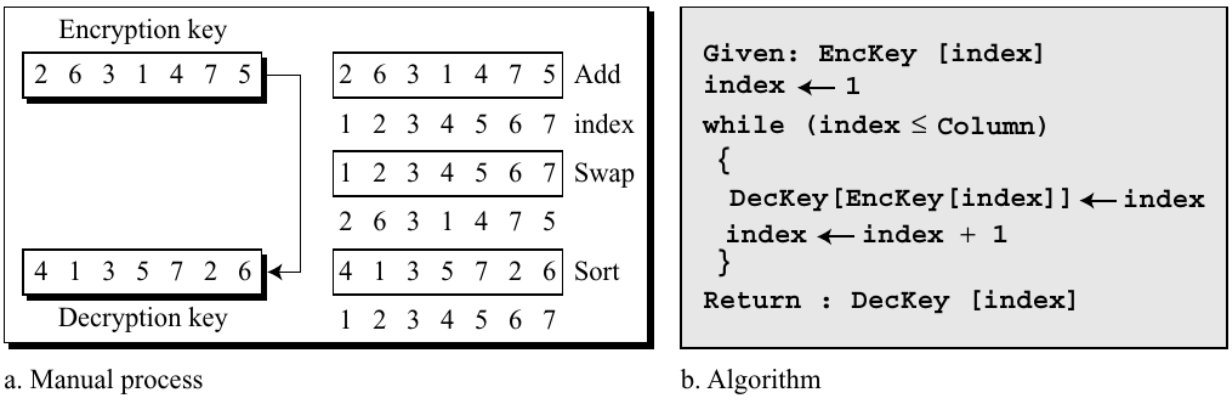
Textbook Figure 3.22: Encryption and decryption keys.

For the five-column example, the encryption key is **(3 1 4 5 2)** and the decryption key is **(2 5 1 3 4)**. The first entry of the encryption key shows that the contents of source column 3 become the contents of destination column 1. The decryption key reverses this rearrangement.

Finding the Inverse Key

The decryption key can be obtained from the encryption key by following the manual process shown in Figure 3.23:

1. Add the column indices to the encryption-key entries.
2. Swap the contents and their indices.
3. Sort the pairs according to the index. The reordered entries give the decryption key.



Textbook Figure 3.23: Manual and algorithmic key inversion.

For the seven-column illustration in the figure, the encryption key **(2 6 3 1 4 7 5)** gives the decryption key **(4 1 3 5 7 2 6)**. The figure also presents the corresponding algorithm for constructing the decryption key.

Using Matrices

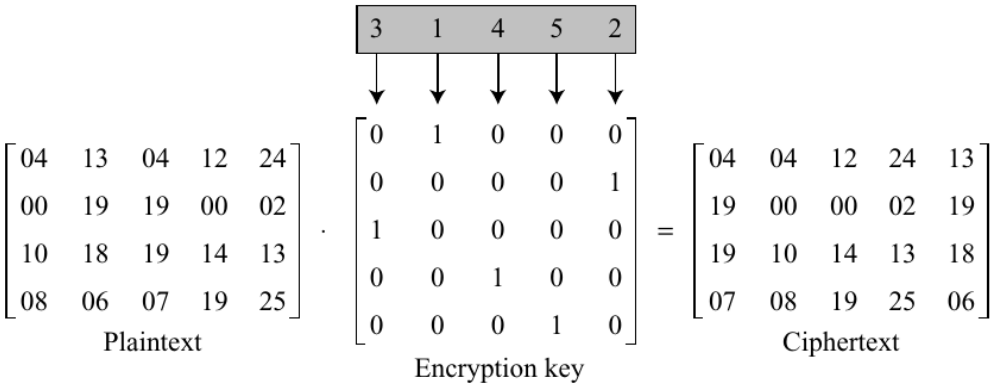
Matrices can represent the encryption and decryption process for a transposition cipher. The plaintext and ciphertext are **l × m matrices** containing numerical values of the characters. The key is an **m × m square matrix**.

- A **permutation matrix** has exactly one 1 in each row and each column; all the other entries are 0.
- Encryption is performed by multiplying the plaintext matrix by the key matrix to obtain the ciphertext matrix.
- Decryption is performed by multiplying the ciphertext matrix by the inverse key matrix to recover the plaintext matrix.

The decryption matrix is the inverse of the encryption matrix. For a permutation matrix, the inverse can be obtained by transposing the encryption-key matrix, that is, swapping its rows and columns.

Example 3.27 — Matrix Representation

In this example, the characters in the plaintext are converted to numerical values from **00 to 25**. Multiplying the **4 × 5** plaintext matrix by the **5 × 5** encryption-key matrix produces the **4 × 5** ciphertext matrix. The multiplication performs the column permutation; writing characters into the matrix and reading them out are still required as part of the complete algorithm.



Textbook Figure 3.24: Representation of the key as a matrix.

The matrix example uses the same key **(3 1 4 5 2)**. The matrix form describes the column rearrangement mathematically, while the table-writing and table-reading steps describe how the text is arranged.

Cryptanalysis of Transposition Ciphers

Transposition ciphers are vulnerable to several kinds of **ciphertext-only attacks**. Since the symbols are rearranged rather than substituted, some properties of the plaintext remain visible in the ciphertext.

Statistical Attack

A transposition cipher does not change the frequency of individual letters; it only reorders them. Therefore, **single-letter frequency analysis** may be useful if the ciphertext is long enough. However, the frequencies of **digrams** (two-letter sequences) and **trigrams** (three-letter sequences) are not preserved. If the single-letter frequencies are preserved but digram and trigram frequencies are not, the cipher may be a transposition cipher.

Brute-Force Attack

An attacker can try all possible keys, but the number of possibilities can be very large: **1! + 2! + 3! + … + L!**, where **L** is the length of the ciphertext. The textbook describes a more practical approach: guess the number of columns. The number of columns must divide the ciphertext length.

For a ciphertext length of 20, the possible column counts are **1, 2, 4, 5, 10, and 20**, because 20 = 1 × 2 × 2 × 5. One column provides no permutation, while 20 columns would give only one row and is considered unlikely.

Example 3.28 — Trying Possible Column Counts

Eve intercepts **EEMYNTAACTTKONSHITZG**, which has length 20. The textbook examines the following possibilities:

a. With two columns, the non-identity permutation produces “ee ym nt aa tc kt no hs ti gz”, which does not form meaningful plaintext.

b. With four columns, there are 4! = 24 permutations. After testing the non-identity permutations, no meaningful plaintext is found.

c. With five columns, there are 5! = 120 permutations. The key **(2 5 1 3 4)** gives **enemyattackstonightz**. Removing the filler z and restoring spaces gives “Enemy attacks tonight”.

Pattern Attack

A keyed transposition cipher can produce repeated patterns in the mapping from ciphertext positions to plaintext positions. In Example 3.28, the textbook gives the following positional sequence:

```
03 08 13 18 01 06 11 16 04 09 14 19 05 10 15 20 02 07 12 17
```

The sequence forms five groups: **(3, 8, 13, 18)**, **(1, 6, 11, 16)**, **(4, 9, 14, 19)**, **(5, 10, 15, 20)**, and **(2, 7, 12, 17)**. The difference between adjacent values in each group is five. If the attacker knows or guesses that there are five columns, organizing the ciphertext into groups of four characters and permuting the groups can provide clues to the plaintext.

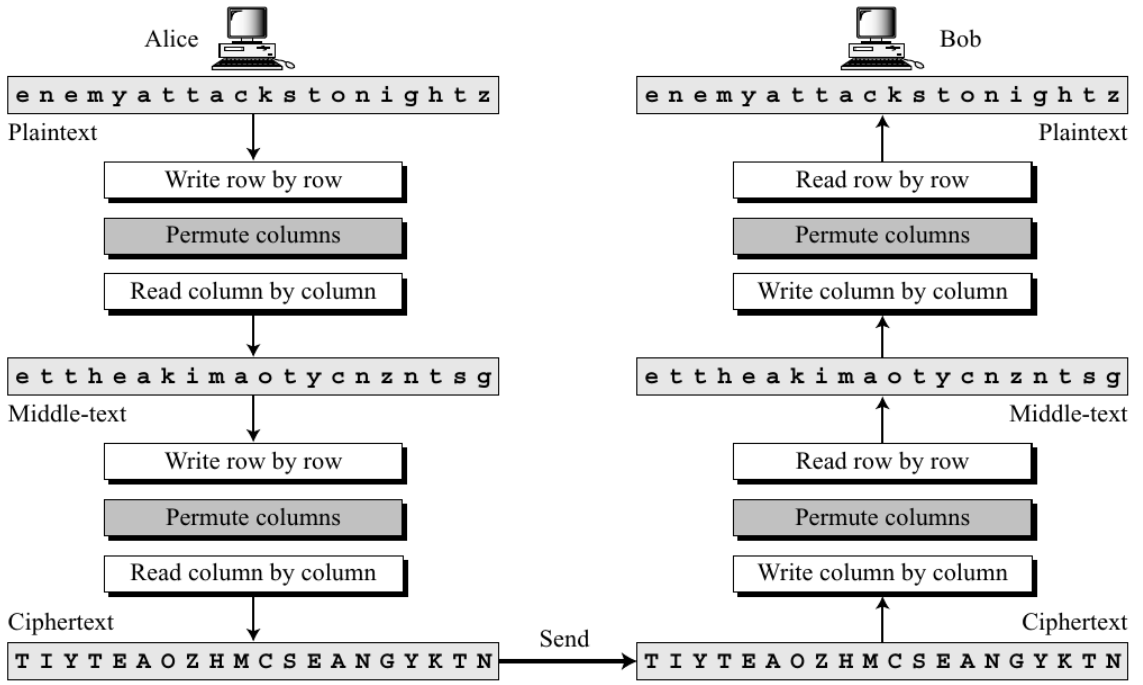
Double Transposition Ciphers

A **double transposition cipher** repeats the columnar transposition encryption process twice. A different key can be used at each stage, but the textbook notes that the same key is normally used. Repeating the rearrangement makes the work of the cryptanalyst more difficult.

Example 3.29 — Double Transposition

The process in Example 3.26 is repeated twice. The first transposition produces the middle text **ettheakimaotycnzntsg**. This middle text is then written row by row, the columns are permuted again, and the resulting table is read column by column. The final ciphertext is:

```
TIYTEAOZHMCSEANGYKTN
```



Textbook Figure 3.25: Double transposition cipher.

Single-letter frequency analysis can still be applied because the letter frequencies are preserved. However, a pattern attack becomes much more difficult. The textbook illustrates this with the positional sequence:

```
13 16 05 07 03 06 10 20 18 04 10 12 01 09 15 17 08 11 19 02
```

Source note: *The printed sequence for this double-transposition example appears to repeat 10 and omit 14. The notes retain the textbook's listed sequence; the point made by the text is that the repeated regularities seen in the single transposition are no longer present.*

Key Points

- Keyless transposition changes the order of symbols through a writing/reading arrangement, such as rail fence or row-wise writing with column-wise reading.
- Keyed transposition rearranges blocks or columns using a permutation key. The inverse key reverses the operation.
- Matrix representation expresses column permutation as multiplication by a permutation matrix; its inverse is its transpose.
- Single-letter frequency is preserved, but digram and trigram frequencies are not. Brute force and pattern analysis can help attack a transposition cipher, while double transposition makes pattern analysis more difficult.

MODERN BLOCK CIPHERS

Notes based on Forouzan, Introduction to Cryptography and Network Security, Chapter 5, Section 5.1

1. Introduction

Traditional symmetric-key ciphers commonly operate on characters. Modern block ciphers are designed for computers and operate on bits. A modern symmetric-key block cipher takes a fixed-size block of plaintext and a key, and produces a ciphertext block of the same size.

If the block contains **n bits** and the key contains **k bits**, the algorithm maps an n-bit plaintext block to an n-bit ciphertext block using the k-bit key. Modern block ciphers are normally **keyed substitution ciphers**, in which the key selects one mapping from the mappings supported by the design.

2. Substitution or Transposition

Substitution

Substitution replaces an input symbol or bit pattern with another. In a modern block cipher, substitutions are performed on groups of bits.

Transposition

Transposition changes the positions of symbols or bits without changing the symbols themselves. Modern block ciphers use permutations as components, but a cipher based only on transposition does not provide the security expected of a modern block cipher. The textbook explains that modern block ciphers are designed as keyed substitution ciphers to resist exhaustive-search attacks.

3. Block Ciphers as Permutation Groups

For a fixed block size of n bits, there are 2■ possible input blocks. A complete one-to-one mapping from these blocks to output blocks is a permutation. A key selects a permitted mapping from the cipher's family of mappings.

- **Full-size key cipher:** the key can select from the complete mapping set being considered.
- **Partial-size key cipher:** the key selects only a subset of all possible mappings.

A practical cipher does not store every possible permutation. Instead, its algorithm constructs a mapping using the key and a sequence of operations.

4. Components of a Modern Block Cipher

Modern block ciphers combine fixed permutation units, substitution units, and other bit-manipulation operations. These components work together to provide diffusion and confusion.

4.1 P-Boxes (Permutation Boxes)

A P-box rearranges the positions of bits according to a fixed mapping. It is similar to a traditional transposition cipher, except that it transposes bits rather than characters. P-boxes are normally keyless: the mapping is predetermined.

- **Straight P-box:** n inputs and n outputs. It rearranges bits without adding or removing any; it is invertible.
- **Compression P-box:** n inputs and m outputs, where m < n. Some input bits are blocked, so the output has fewer bits.
- **Expansion P-box:** n inputs and m outputs, where m > n. Some input bits are connected to more than one output, so selected bits are repeated.

A straight P-box can be reversed by applying its inverse permutation. Compression and expansion P-boxes, by themselves, are generally not invertible because bits can be discarded or duplicated.

Straight	Compression	Expansion
n inputs → n outputs Reorders bits	n inputs → m outputs m < n; discards some bits	n inputs → m outputs m > n; repeats some bits

4.2 S-Boxes (Substitution Boxes)

An S-box is a small substitution unit. It accepts a group of input bits and produces an output bit group according to a predefined mapping. Unlike a P-box, which changes bit positions, an S-box changes bit values.

- The number of input bits and output bits need not be equal.
- S-boxes introduce nonlinearity, an important property in modern block ciphers.
- An S-box can be represented using a table or a mathematical mapping.

S-boxes and P-boxes have complementary roles: S-boxes change values, while P-boxes spread the resulting bits to different positions.

4.3 Exclusive-OR (XOR)

XOR is a bitwise operation frequently used to combine data with a key or round key. It gives 1 when the input bits differ and 0 when they are the same.

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

XOR is reversible when one operand is known: if C = P XOR K, then P = C XOR K. The textbook relates XOR to addition and subtraction in the binary field GF(2).

4.4 Circular Shift and Swap

- **Circular shift:** bits shifted off one end re-enter at the other end. Shifts can be left or right.
- **Swap:** exchanges two parts of a word. Swapping two equal halves is a special case of circular shifting by half the word length.

4.5 Split and Combine

Split divides a word into smaller parts, such as dividing an 8-bit word into two 4-bit halves. **Combine** joins those parts back into one word. These operations help organise data for processing by other cipher components.

5. Product Ciphers

Claude Shannon introduced the idea of a **product cipher**: a complex cipher combining substitution, permutation and other operations. Modern block ciphers are product ciphers because they combine multiple transformations to achieve stronger security properties.

A typical round may perform the following steps:

- Mix the current data with a round key, commonly using XOR. This is called key mixing or whitening.
- Pass groups of bits through S-boxes to substitute their values.
- Pass the result through a P-box to permute the bit positions.
- Repeat the transformations for multiple rounds, using round keys generated from the original cipher key.

The intermediate result between rounds is called the **middle text**. A **round** is one iteration of the transformations. A **key schedule** or **key generator** produces the keys used in different rounds.

Plaintext block	→	**Key mixing** XOR with K■	→	**S-boxes** Substitute bits
		↓		↓
Ciphertext	←	**P-box** Permute bits	←	**Next round** Use next round key

6. Diffusion and Confusion

Diffusion

Diffusion hides the relationship between plaintext and ciphertext. A change in one plaintext bit should affect several ciphertext bits after the rounds. This makes it difficult to infer the plaintext from patterns in the ciphertext.

Confusion

Confusion hides the relationship between the key and ciphertext. A change in a key bit should affect many ciphertext bits, making the key-to-ciphertext relationship difficult to analyse.

S-boxes, P-boxes, key mixing and repeated rounds work together to create diffusion and confusion. Practical ciphers generally use sufficiently large blocks, multiple S-boxes and multiple rounds to strengthen these properties.

7. Two Classes of Product Ciphers

7.1 Feistel Ciphers

Feistel ciphers use a structure that can include invertible and noninvertible components. The overall cipher remains decryptable because of the way the rounds are arranged; decryption uses the round keys in the appropriate order. **DES** is a textbook example of a Feistel cipher.

7.2 Non-Feistel Ciphers

Non-Feistel ciphers use only invertible components in their overall transformation. Decryption reverses the transformations using their inverse operations. **AES** is a textbook example of a non-Feistel cipher.

Main distinction: a Feistel structure can employ noninvertible round components, whereas a non-Feistel structure uses invertible components.

8. Attacks on Block Ciphers

- **Differential cryptanalysis:** studies how differences in selected plaintext pairs influence differences in the resulting ciphertexts.
- **Linear cryptanalysis:** searches for approximate linear relationships among plaintext bits, ciphertext bits and key bits.

Modern block cipher design aims to make such relationships difficult to exploit. Diffusion, confusion, suitable S-boxes and enough rounds help resist these attacks. Security depends on the complete design and key size, not merely on the presence of individual components.

9. Quick Revision Points

- A modern block cipher encrypts a fixed-size n-bit plaintext block into an n-bit ciphertext block using a k-bit key.
- P-boxes permute bit positions; S-boxes substitute bit patterns.
- Straight P-box: same input/output size; compression: fewer outputs; expansion: more outputs.
- XOR combines bit strings and is reversible when one operand is known.
- Circular shift rotates bits; swap exchanges parts; split and combine divide and rejoin words.
- Product ciphers combine S-boxes, P-boxes, key mixing and rounds.
- Diffusion hides plaintext–ciphertext relationships; confusion hides key–ciphertext relationships.
- Feistel ciphers may use noninvertible components; non-Feistel ciphers use invertible components.
- Differential and linear cryptanalysis are important attacks considered in this section.

10. Key Terms

Block cipher: symmetric-key cipher processing fixed-size blocks. **P-box:** permutation box. **S-box:** substitution box. **Product cipher:** cipher combining multiple operations. **Round:** one iteration of transformations. **Key schedule:** procedure that generates round keys. **Diffusion:** hides plaintext–ciphertext relationships. **Confusion:** hides key–ciphertext relationships. **Feistel cipher:** product-cipher structure that may use noninvertible components. **Non-Feistel cipher:** product-cipher structure using invertible components.

Source: Behrouz A. Forouzan, Introduction to Cryptography and Network Security, Chapter 5, Section 5.1, “Modern Block Ciphers” (with related chapter-summary points).

Components of modern block ciphers

1. Introduction

Modern block ciphers are generally keyed substitution ciphers. A key limits which input-to-output mappings can be used. Instead of being built as one indivisible operation, a modern block cipher combines several smaller components so that the complete cipher can provide diffusion and confusion.

The main components covered here are permutation boxes (P-boxes), substitution boxes (S-boxes), and other operations such as XOR, complement, circular shift, swap, split and combine. Several of these are assembled into a product cipher.

2. P-boxes (Permutation boxes)

A P-box performs a transposition of bits: it changes the positions of bits rather than substituting their values. The three common forms are straight, compression and expansion P-boxes.

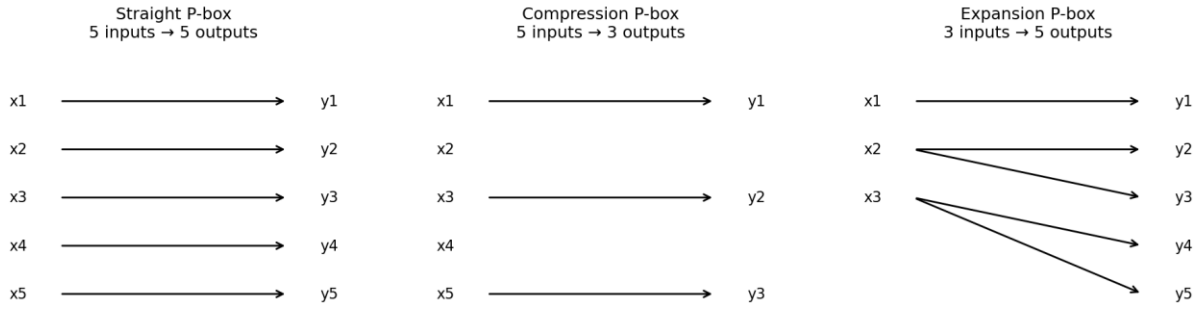


Diagram: the three common P-box forms (schematic).

Type	Input/output size	What happens	Typical use
Straight	n → n	Reorders all bits; no bit is removed or duplicated.	Permutation while preserving word length.
Compression	n → m, where m < n	Some input bits are blocked and do not appear at the output.	Permute bits and reduce the number of bits for a later stage.
Expansion	n → m, where m > n	Some input bits are connected to more than one output position.	Permute bits and increase the number of bits for a later stage.

2.1 Straight P-boxes

A straight P-box with n inputs and n outputs is a permutation. There are n! possible mappings. For example, a 3 × 3 straight P-box has 3! = 6 possible mappings.

Mapping number	Output positions receive input bits
1	[1, 2, 3]
2	[1, 3, 2]
3	[2, 1, 3]
4	[2, 3, 1]
5	[3, 1, 2]
6	[3, 2, 1]

P-boxes are usually keyless: the mapping is fixed in advance. A hardware implementation can be wired to perform the mapping; a software implementation can use a permutation table. In a table, each entry tells which input bit supplies the output at that position.

Example: a 64-bit straight P-box can be described by a table of 64 entries. If the first entry is 58, output bit 1 comes from input bit 58. If the last entry is 7, output bit 64 comes from input bit 7.

Example 1 — Design an 8 × 8 straight P-box

Requirement: move input bits 4 and 5 to output positions 1 and 8, while keeping the relative order of the other bits.

Permutation table: [4, 1, 2, 3, 6, 7, 8, 5].

Output position	1	2	3	4	5	6	7	8
Source input bit	4	1	2	3	6	7	8	5

Thus, output 1 takes input 4 and output 8 takes input 5; the other listed bits remain in their relative order.

2.2 Compression P-boxes

A compression P-box has n inputs and m outputs, with m < n. Some inputs are blocked, so they do not reach the output. Its permutation table has m entries selected from the input positions 1 through n; omitted positions are the blocked inputs.

Example: a 32 × 24 compression P-box has 24 table entries. In the example table below, input positions 7, 8, 9, 15, 16, 23, 24 and 25 are omitted, so those bits are discarded.

Example 32 × 24 compression table (24 entries)
01 02 03 21 22 26 27 28 29 13 14 17
18 19 20 04 05 06 10 11 12 30 31 32

Compression is useful when the next stage needs fewer bits.

2.3 Expansion P-boxes

An expansion P-box has n inputs and m outputs, with m > n. Some input bits are mapped to more than one output position. Its table has m entries, with m − n repeated entries.

Example: in a 12 × 16 expansion P-box, the table has 16 entries. Input bits 1, 3, 9 and 12 each appear twice.

Example 12 × 16 expansion table
01 09 10 11 12 01 02 03 03 04 05 06 07 08 09 12

Expansion is useful when a later stage needs more bits.

2.4 Invertibility of P-boxes

A straight P-box is invertible. Decryption uses the inverse permutation table, which reverses the original mapping.

Example 2 — Inverting a permutation table

Original table: [6, 3, 4, 5, 2, 1]. Add the output indices, exchange the table contents and indices, then sort by the indices. The inverse table is [6, 5, 2, 3, 4, 1].

Original output position	1	2	3	4	5	6
Input bit used	6	3	4	5	2	1

Inverse output position	1	2	3	4	5	6
Input bit used	6	5	2	3	4	1

Compression and expansion P-boxes are not invertible by themselves. Compression loses information because a discarded input bit could have been either 0 or 1. Expansion duplicates an input bit, so the output alone does not always identify which repeated position corresponds to the original input.

A compression P-box is not the inverse of an expansion P-box, and an expansion P-box is not the inverse of a compression P-box. Some cipher designs can still use these components if their overall construction restores reversibility in another way.

P-box type	Invertible on its own?	Reason
Straight	Yes	Every input bit is preserved exactly once.
Compression	No	One or more input bits are lost.
Expansion	No	Some inputs are duplicated, so the mapping is not one-to-one.

3. S-boxes (Substitution boxes)

An S-box is a small substitution unit. It maps an n-bit input word to an m-bit output word; n and m do not have to be equal. S-boxes may be keyed or keyless, but modern block ciphers commonly use fixed, keyless mappings.

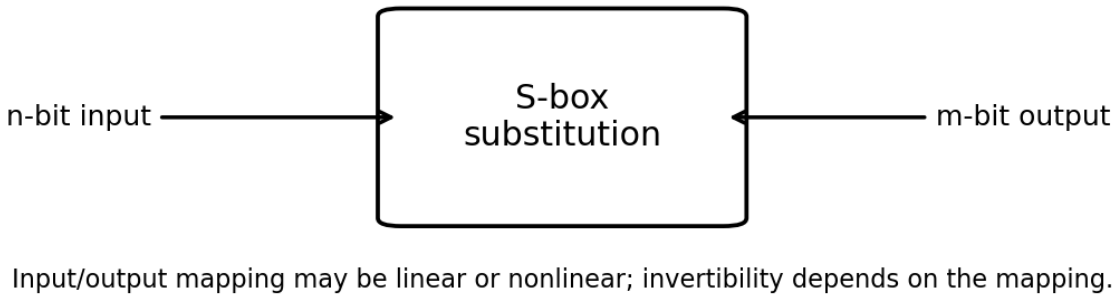


Diagram: general S-box input/output relationship (schematic).

3.1 Linear and nonlinear S-boxes

The input/output relationship of an S-box can be expressed as equations: each output bit y_i is a function of the input bits $x_1, x_2, \ldots, x_n$. In a linear S-box, each output bit is an XOR of selected input bits. In a nonlinear S-box, at least one output cannot be expressed in that linear form.

Example 3 — Linear S-box with three input bits and two output bits

Let $y_1 = x_1 \oplus x_2 \oplus x_3$ and $y_2 = x_1$. This is linear because each output is an XOR of input bits. The same relation can be represented by a matrix over GF(2):

Output	Equation
y_1	$x_1 \oplus x_2 \oplus x_3$
y_2	x_1

Matrix form: $[y_1, y_2]^T = [[1, 1, 1], [1, 0, 0]]\ [x_1, x_2, x_3]^T$, with operations over GF(2).

Example 4 — Nonlinear S-box

For three inputs and two outputs, let $y_1 = (x_1)^3 \oplus x_2$ and $y_2 = (x_1)^2 \oplus (x_1x_2) \oplus x_3$, where the operations are in GF(2). This is nonlinear because the input/output relationship cannot be expressed entirely as a linear XOR combination of the input bits.

Example 5 — Read a 3 × 2 S-box table

The leftmost input bit selects the row; the two rightmost bits select the column. The cell at that row and column gives the two output bits.

Leftmost input bit	Column 00	Column 01	Column 10	Column 11
0	00	10	01	11
1	10	00	11	01

Examples: input 010 selects row 0 and column 10, giving output 01. Input 101 selects row 1 and column 01, giving output 00.

3.2 Invertibility of S-boxes

An S-box may or may not be invertible. For a substitution to be invertible, the input and output lengths must be equal and every output must correspond to exactly one input.

Example 6 — Inverse S-box tables

The following pair of 3-bit tables are inverses. The left table is used for encryption and the right table for decryption. Read the leftmost bit as the row and the next two bits as the column.

Input row	Column 00	Column 01	Column 10	Column 11
Encryption, row 0	011	101	111	100
Encryption, row 1	000	010	001	110

Input row	Column 00	Column 01	Column 10	Column 11
Decryption, row 0	100	110	101	000
Decryption, row 1	011	001	111	010

For example, encryption input 001 gives 101; applying the decryption table to 101 gives 001. This confirms the inverse relationship.

4. Other components

4.1 Exclusive-or (XOR)

XOR is important in block ciphers. In the field GF(2^n), addition and subtraction are both performed using XOR.

Important properties:

- Closure: XOR of two n-bit words produces another n-bit word.
- Associativity: $x \oplus (y \oplus z) = (x \oplus y) \oplus z$.
- Commutativity: $x \oplus y = y \oplus x$.
- Identity: $x \oplus 00\ldots0 = x$.
- Self-inverse: $x \oplus x = 00\ldots0$.

The identity and self-inverse properties are especially useful in cipher design.

4.2 Complement

Complement flips every bit: 0 becomes 1 and 1 becomes 0. If $\overline{x}$ denotes the complement of x, then $x \oplus \overline{x} = 11\ldots1$ and $x \oplus 11\ldots1 = \overline{x}$.

4.3 Invertibility of XOR

XOR has two inputs, so reversing it requires one input to be known. When the key k is fixed for encryption and decryption, XOR is self-invertible.

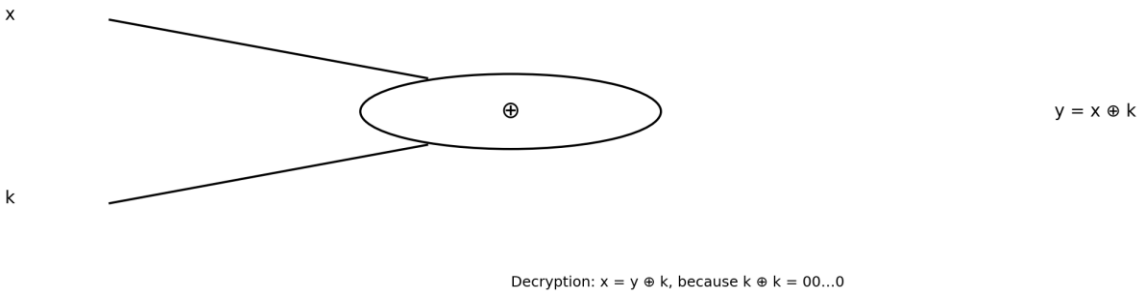


Diagram: XOR key mixing and recovery.

If y = x ⊕ k, then x = y ⊕ k, because k ⊕ k = 00…0.

4.4 Circular shift

A circular shift moves bits around a word. In a circular left shift by k positions, the leftmost k bits wrap around to the right end. In a circular right shift, the rightmost k bits wrap around to the left end.

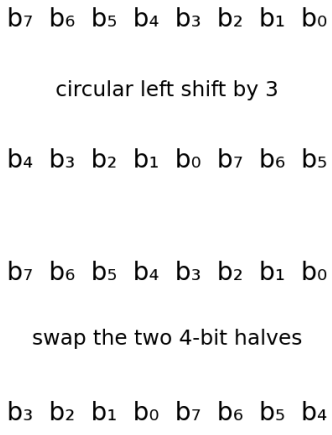


Diagram: 8-bit circular shift by three positions and a half-word swap.

Circular shifts mix the positions of bits and can help conceal patterns. The shift amount is normally fixed rather than used as a key.

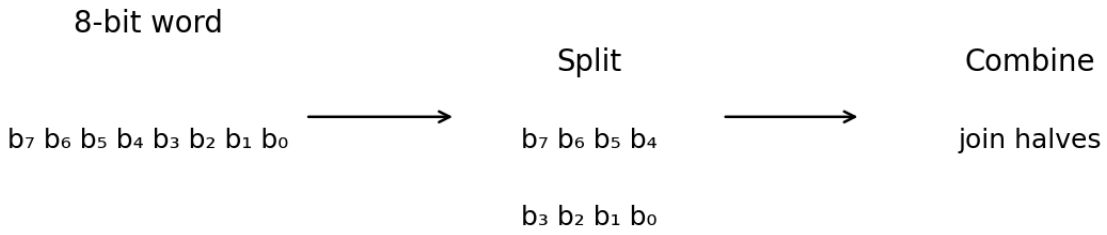
Invertibility: a circular left shift is reversed by a circular right shift of the same amount, and vice versa. Shifts are modulo n: shifting by n positions, or by 0 positions, leaves the word unchanged; a shift greater than n is equivalent to shifting by k mod n. Repeated shifts combine into one shift.

4.5 Swap

Swap is a special circular shift in which k = n/2, so it applies only to words with an even number of bits. It exchanges the two halves of the word. Shifting left or right by half the word length gives the same result, and applying swap twice restores the original word.

4.6 Split and combine

Split divides a word—normally at its midpoint—into two equal-length parts. Combine concatenates two equal-length parts to recreate the full word. They are inverse operations: splitting followed by combining the halves in the same order restores the original word.



Split and combine undo each other when the halves are kept in order.

Diagram: splitting and combining an 8-bit word.

5. Product ciphers

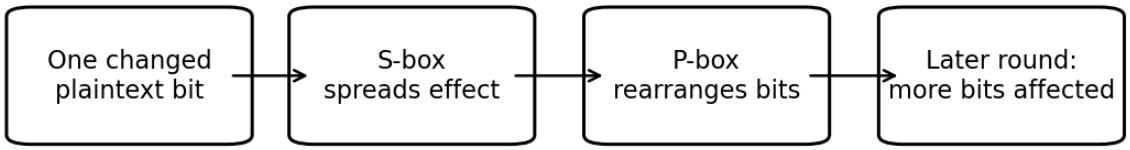
A product cipher combines substitution, permutation and other basic components into a more complex cipher. The design uses several stages rather than relying on a single transformation.

5.1 Diffusion and confusion

Diffusion hides the relationship between plaintext and ciphertext. A change in one plaintext bit should spread so that many ciphertext bits are affected. This makes it difficult to use statistical patterns in the ciphertext to infer the plaintext.

Confusion hides the relationship between the key and the ciphertext. The influence of individual key bits should be distributed and complicated, so that the ciphertext does not reveal a simple relationship to the key.

Property	What it hides	Typical way it is encouraged
Diffusion	Plaintext ↔ ciphertext relationship	Combining S-box substitution with P-box permutation over rounds.
Confusion	Key ↔ ciphertext relationship	Key mixing and nonlinear substitutions across rounds.



Diffusion: a small plaintext change spreads across many ciphertext bits.

Confusion: the effect of key bits on ciphertext is made complex.

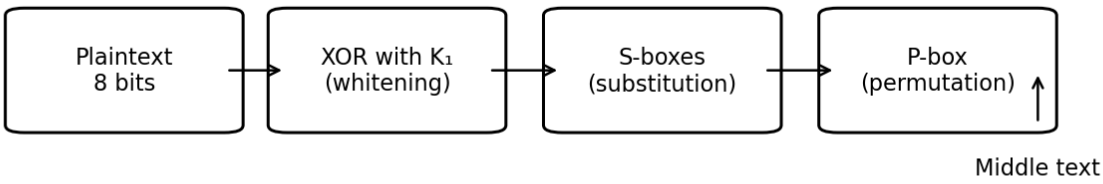
Diagram: conceptual spread of a bit change and the roles of diffusion and confusion.

Diffusion and confusion can be achieved using an iterated product cipher, where a round of transformations is repeated several times. The intermediate result between rounds is called the middle text.

5.2 Two-round product cipher

A simple illustrative product cipher uses an 8-bit plaintext and two rounds. Each round performs three main transformations:

1. Key mixing (whitening): XOR the 8-bit word with an 8-bit round key (K_1 in round 1 and K_2 in round 2).
2. Substitution: split the result into four 2-bit groups and pass them through four S-boxes. Each S-box changes the bit values according to its fixed mapping.
3. Permutation: send the S-box outputs through a P-box. The bit positions are rearranged so that the next round's S-boxes receive different combinations of bits.



Round 2 repeats key mixing → S-box substitution → P-box permutation

After the final round: ciphertext

Diagram: simplified two-round product-cipher flow (schematic).

The round-key generator derives round keys from the cipher key K. After the first round, the result is the middle text; after the final round, the result is the ciphertext. Decryption reverses the overall transformations using the corresponding inverse operations and round keys.

5.3 How diffusion spreads through rounds

In the two-round example, changing plaintext bit 8 affects bits 7 and 8 after XOR with the first round key and passage through S-box 4. The P-box sends these to middle-text bits 2 and 4. In round 2, bit 2 influences bits 1 and 2 through S-box 1, which the P-box sends to ciphertext bits 6 and 1. Bit 4 influences bits 3 and 4; the P-box sends these to ciphertext bits 3 and 7. Therefore, the original change in plaintext bit 8 affects ciphertext bits 1, 3, 6 and 7.

Following the transformations in reverse also shows that a ciphertext bit can depend on multiple plaintext bits.

5.4 How confusion is produced

In the same example, ciphertext bits 1, 3, 6 and 7 are influenced by key bit 8 of K_1 and key bits 2 and 4 of K_2. Tracing the transformations backward shows that each round-key bit can affect several ciphertext bits. The relationship between the key and the ciphertext is therefore obscured.

5.5 Practical ciphers

Practical designs improve diffusion and confusion by using larger data blocks, more S-boxes and more rounds. Additional rounds spread the influence of plaintext bits across more of the ciphertext and increase the number of round keys involved, making the relationship between plaintext, ciphertext and key more difficult to recognize.

Attacks on Block Ciphers

1. Attacks on Block Ciphers – Overview

- Attacks on **traditional ciphers** can also be used on **modern block ciphers**, but today's block ciphers resist most of the attacks.
- For example, **brute-force attack** on the key is usually **infeasible** because the keys normally are very large.
- However, some **new attacks** on block ciphers have been devised that are based on the **structure of the modern block cipher**.
- These attacks use **differential** and **linear cryptanalysis** techniques.

2. Differential Cryptanalysis

- Introduced by **Eli Biham** and **Adi Shamir**.
- It is a **chosen-plaintext attack**: Eve somehow accesses Alice's computer, submitting chosen plaintext and obtaining the corresponding ciphertext.
- **Goal:** to find **Alice's cipher key**.

2.1 Algorithm Analysis

- Before Eve uses the chosen-plaintext attack, she needs to **analyze the encryption algorithm** to collect some **plaintext–ciphertext relationships**.
- Eve does not know the cipher key. However, some ciphers have **weaknesses in their structures** that allow Eve to find a relationship between the **plaintext differences** and **ciphertext differences** without knowing the key.

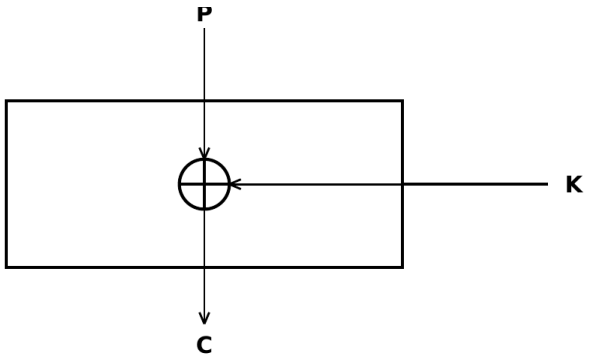
2.2 Example 5.13

Assume the cipher is made of only **one exclusive-or operation** (Figure 5.18). Without knowing the key, Eve can easily find the relationship between **plaintext differences** and **ciphertext differences**. By plaintext difference we mean $P_1 \oplus P_2$, and by ciphertext difference, $C_1 \oplus C_2$. The following proves that $C_1 \oplus C_2 = P_1 \oplus P_2$:

$$C_1 = P_1 \oplus K \quad C_2 = P_2 \oplus K \quad \rightarrow \quad C_1 \oplus C_2 = P_1 \oplus K \oplus P_2 \oplus K = P_1 \oplus P_2$$

However, this example is **very unrealistic**; modern block ciphers are not so simple.

Figure 5.18 *Diagram for Example 5.13*



2.3 Example 5.14

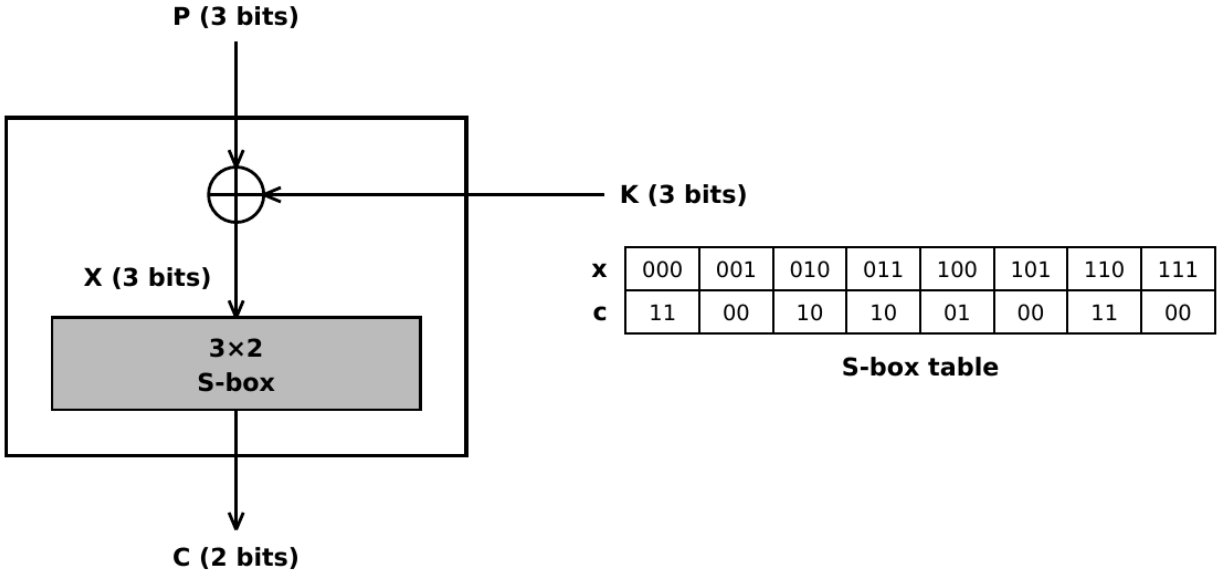
We **add one S-box** to Example 5.13 (Figure 5.19).

- Plaintext P (**3 bits**) is XORed with key K (**3 bits**) to give X (3 bits).
- X passes through a **3×2 S-box** giving ciphertext C (**2 bits**).

S-box table (Figure 5.19):

x	000	001	010	011	100	101	110	111
c	11	00	10	10	01	00	11	00

Figure 5.19 *Diagram for Example 5.14*



Although the effect of the key is still **canceled** when we use differences between two X's and two P's ($X_1 \oplus X_2 = P_1 \oplus P_2$), the existence of the **S-box prevents Eve from finding a definite relationship** between the plaintext differences and the ciphertext differences. However, she can create a **probabilistic relationship**.

Eve can make **Table 5.4**, which shows, for each plaintext difference, how many ciphertext differences the cipher may create. Note that the table is made from information about the S-box input/output table in Figure 5.19 because $P_1 \oplus P_2 = X_1 \oplus X_2$.

Table 5.4 – Differential input/output for the cipher in Example 5.14

$P_1 \oplus P_2 \setminus C_1 \oplus C_2$	00	01	10	11
000	8			
001	2	2		4
010	2	2	4	
011		4	2	2
100	2	2	4	
101		4	2	2
110	4		2	2
111			2	6

- Because the key size is **3 bits**, there can be **eight cases** for each difference in the input.

- If the input difference is $(000)_2$, the output difference is always $(00)_2$.
- If the input difference is $(001)_2$, there are **two cases** of $(00)_2$ output difference, **two cases** of $(01)_2$, and **four cases** of $(11)_2$ output difference.

2.4 Example 5.15

The **heuristic result** of Example 5.14 can create **probabilistic information** for Eve as shown in **Table 5.5**. The entries show the **probabilities of occurrences**. Entries with **zero probability will never occur**.

Table 5.5 – Differential distribution table for Example 5.15

$P_1 \oplus P_2 \setminus C_1 \oplus C_2$	00	01	10	11
000	1	0	0	0
001	0.25	0.25	0	0.50
010	0.25	0.25	0.50	0
011	0	0.50	0.25	0.25
100	0.25	0.25	0.50	0
101	0	0.50	0.25	0.25
110	0.50	0	0.25	0.25
111	0	0	0.25	0.75

- Eve now has a great deal of information to start her attack.
- The table shows that the probabilities are **not distributed uniformly** because of the weakness in the structure of the S-box.
- Table 5.5 is sometimes referred to as the **differential distribution table** or **XOR profile**.

2.5 Launching a Chosen-Plaintext Attack

- After the analysis, which can be done **once** and kept for future uses as long as the structure of the cipher does not change, Eve can choose the plaintexts for attacks.
- The differential probability distribution table (Table 5.5) helps Eve choose plaintexts that have the **highest probability** in the table.

2.6 Guessing the Key Value

- After launching some attacks with appropriate chosen plaintexts, Eve can find some **plaintext–ciphertext pairs** that allow her to guess the value of the key.
- This step **starts from C and works toward P**.

2.7 Example 5.16

Looking at Table 5.5, Eve knows that if $P_1 \oplus P_2 = 001$, then $C_1 \oplus C_2 = 11$ with the probability of **0.50 (50 percent)** (chosen-plaintext attack). She tries $C_1 = 00$ and gets $P_1 = 010$ (chosen-plaintext attack). She also tries $C_2 = 11$ and gets $P_2 = 011$ (another chosen-plaintext attack). Now she works **backward**, based on the first pair, P_1 and C_1.

$$C_1 = 00 \rightarrow X_1 = 001 \text{ or } X_1 = 111$$

$$\text{If } X_1 = 001 \rightarrow K = X_1 \oplus P_1 = 011 \qquad \text{If } X_1 = 111 \rightarrow K = X_1 \oplus P_1 = 101$$

Using the **second pair**, P_2 and C_2:

$$C_2 = 11 \rightarrow X_2 = 000 \text{ or } X_2 = 110$$

$$\text{If } X_2 = 000 \rightarrow K = X_2 \oplus P_2 = 011 \qquad \text{If } X_2 = 110 \rightarrow K = X_2 \oplus P_2 = 101$$

- The two tests confirm that **K = 011 or K = 101**.
- Although Eve is not sure what the exact value of the key is, she knows that the **rightmost bit is 1** (the common bit between the two values).
- More attacks, with the assumption that the rightmost bit in the key is 1, can reveal more bits in the key.

2.8 General Procedure

Modern block ciphers have more complexity than discussed above; in addition, they are made from different **rounds**. Eve can use the following strategy:

1. Because each round is the same, Eve can create a **differential distribution table (XOR profile)** for each S-box and combine them to create the distribution for each round.
2. Assuming that each round is **independent** (a fair assumption), Eve can create a distribution table for the whole cipher by **multiplying the corresponding probabilities**.
3. Eve now can make a list of plaintexts for attacks based on the distribution table in step 2. Note that the table in step 2 only helps Eve choose a **smaller number of plaintext–ciphertext pairs**.
4. Eve chooses a ciphertext and finds the corresponding plaintext. She then analyzes the result to find some bits in the key.
5. Eve repeats step 4 to find more bits in the key.
6. After finding enough bits in the key, Eve can use a **brute-force attack** to find the whole key.

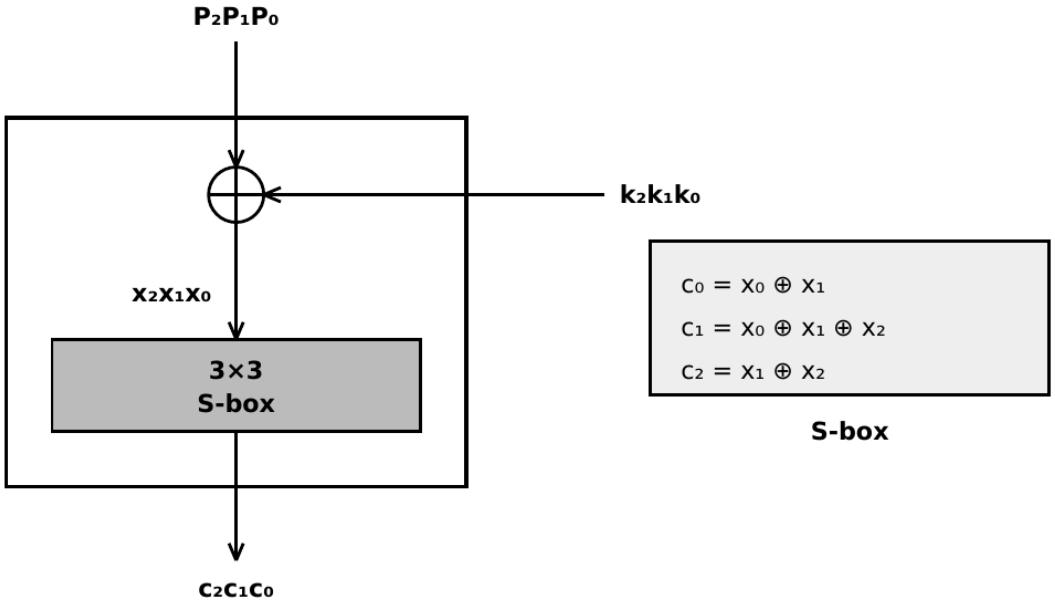
Differential cryptanalysis is based on a nonuniform differential distribution table of the S-boxes in a block cipher.

Note: A more detailed differential cryptanalysis is given in Appendix N.

3. Linear Cryptanalysis

- Presented by **Mitsuru Matsui** in **1993**.
- The analysis uses **known-plaintext attacks** (versus the chosen-plaintext attacks in differential cryptanalysis).
- The thorough discussion of this attack is based on some probability concepts that are beyond the scope of this book.
- Main idea: assume the cipher is made of a **single round** (Figure 5.20), where c_0, c_1, and c_2 represent the three bits in the output and x_0, x_1, and x_2 represent the three bits in the input of the S-box.
- The S-box is a **linear transformation** in which each output is a linear function of input, as discussed earlier in this chapter. With this linear component, we can create **three linear equations** between plaintext and ciphertext bits.

Figure 5.20 *A simple cipher with a linear S-box*



Three linear equations (between plaintext and ciphertext):

$$c_0 = P_0 \oplus k_0 \oplus P_1 \oplus k_1$$
$$c_1 = P_0 \oplus k_0 \oplus P_1 \oplus k_1 \oplus P_2 \oplus k_2$$
$$c_2 = P_1 \oplus k_1 \oplus P_2 \oplus k_2$$

Solving for three unknowns, we get:

$$k_1 = (P_1) \oplus (c_0 \oplus c_1 \oplus c_2)$$
$$k_2 = (P_2) \oplus (c_0 \oplus c_1)$$
$$k_0 = (P_0) \oplus (c_1 \oplus c_2)$$

- This means that **three known-plaintext attacks** can find the values of k_0, k_1, and k_2.
- However, real block ciphers are not as simple as this one; they have **more components** and the **S-boxes are not linear**.

3.1 Linear Approximation

- In some modern block ciphers, it may happen that some S-boxes are **not totally nonlinear**; they can be **approximated, probabilistically**, by some **linear functions**.
- In general, given a cipher with plaintext and ciphertext of **n bits** and a key of **m bits**, we are looking for some equations of the form:

$$(k_0 \oplus k_1 \oplus \ldots \oplus k_x) = (P_0 \oplus P_1 \oplus \ldots \oplus P_y) \oplus (c_0 \oplus c_1 \oplus \ldots \oplus c_z)$$

- where $1 \le x \le m$, $1 \le y \le n$, and $1 \le z \le n$.
- The bits in the intercepted plaintext and ciphertext can be used to find the key bits.
- To be effective, each equation should hold with probability **1/2 + ε**, where **ε is called the bias**.
- An equation with a **larger ε** is **more effective** than one with smaller ε.

Note: A more detailed linear cryptanalysis is given in Appendix N.

4. Quick Comparison

Point	Differential Cryptanalysis	Linear Cryptanalysis

Introduced by	Eli Biham and Adi Shamir	Mitsuru Matsui (1993)
Attack type	Chosen-plaintext attack	Known-plaintext attack
Based on	Nonuniform differential distribution table (XOR profile) of S-boxes	Linear approximation of S-boxes with bias ε (probability 1/2 + ε)
Goal	Find the cipher key (bits, then brute-force the rest)	Find key bits using linear equations of plaintext, ciphertext and key bits

TWO CLASSES OF PRODUCT CIPHERS

Transposition Ciphers · Introduction to Modern Symmetric-Key Ciphers

1. Introduction to Product Ciphers

A **product cipher** is a complex cipher that combines substitution, permutation and other simple components (such as the exclusive-or with a key) into several repeated stages. The idea was introduced by Shannon to obtain two properties that no single operation can provide: **diffusion** and **confusion**. A substitution alone only replaces values, a permutation alone only reorders positions, and a lone XOR with a key is easy to analyse; combining them over many rounds produces a cipher far stronger than any of its parts. Modern block ciphers are all product ciphers.

Property	What it hides	Meaning	Mainly achieved by
Diffusion	Plaintext ↔ ciphertext relationship	Each ciphertext symbol depends on some or all plaintext symbols, so a one-symbol change in the plaintext affects several or all ciphertext symbols.	P-boxes (permutation) working with S-boxes, repeated over rounds
Confusion	Key ↔ ciphertext relationship	The influence of the key bits on the ciphertext is made so complicated that the ciphertext reveals no simple relation to the key.	Key mixing (XOR) and nonlinear S-boxes

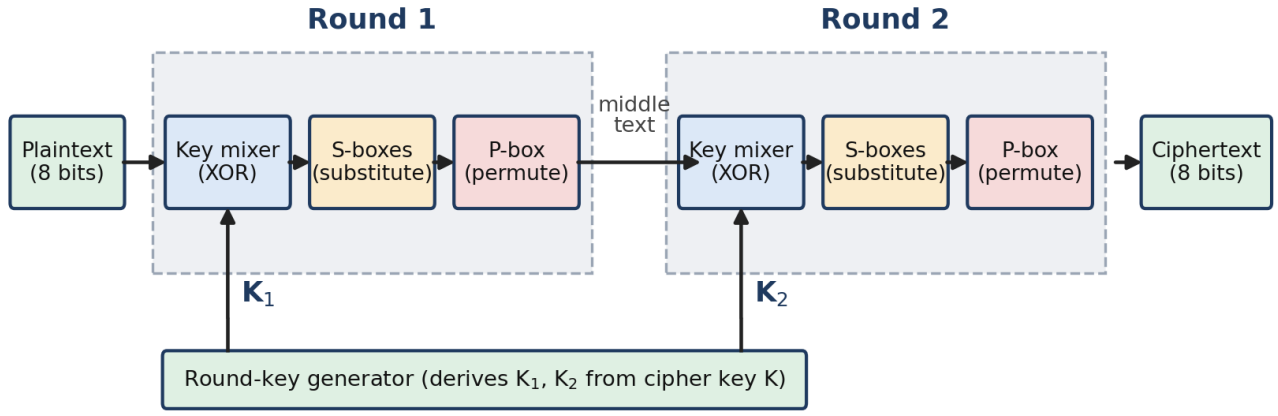


Figure 1. Operation flow of a two-round product cipher on an 8-bit block: each round performs key mixing, S-box substitution and P-box permutation.

Rounds. Diffusion and confusion are obtained with an *iterated product cipher*, in which each iteration, called a **round**, is a combination of S-boxes, P-boxes and other components. A key schedule (round-key generator) derives a different round key K_i for every round from the cipher key K. In an N-round cipher the plaintext is encrypted N times to give the ciphertext, and the ciphertext is decrypted N times to give the plaintext. The text produced between rounds is called the **middle text**.

2. Main Components of Product Ciphers

Component	What it does	Contribution	Invertible?
Key mixer (XOR)	XORs the data with the round key (whitening): $y = x \oplus K_i$.	Hides the data with the key (confusion).	Yes – self-invertible: $(x \oplus k) \oplus k = x$.
S-box	Replaces an n-bit input word by an m-bit output word using a fixed table.	Nonlinearity and confusion.	Only if n = m and the mapping is one-to-one.
P-box	Changes the positions of bits; bit values are unchanged.	Diffusion.	Straight P-box: yes. Compression / expansion: no.
Swap, split, combine	Swap exchanges the two halves of a block; split divides a block into halves; combine joins them.	Provide the left/right halves used by Feistel ciphers.	Yes – swap is self-inverse; split and combine undo each other.
Round	One complete pass of the above components with its own round key K_i.	Repeated N times to spread the effect of every bit.	Invertible when its components (or its Feistel structure) are.

XOR is central because $x \oplus x = 00\ldots0$ and $x \oplus 00\ldots0 = x$; this self-inverse property lets the same key value cancel itself during decryption.

3. The Two Classes of Product Ciphers

All modern block ciphers are product ciphers, but they are divided into **two classes** according to the kind of components they are allowed to use. **Feistel ciphers** may use components that are not invertible, and still remain reversible through their overall structure. **Non-Feistel ciphers** use only invertible components, so every component of encryption has a matching inverse in decryption. **DES** is the standard example of a Feistel cipher (16 Feistel rounds), and **AES** is the standard example of a non-Feistel cipher.

4. Feistel Ciphers

Feistel designed a very intelligent cipher that has been used for decades. A Feistel cipher can contain three types of components: **self-invertible** (e.g. XOR, swap), **invertible** (e.g. straight P-box, invertible S-box) and **noninvertible** (e.g. compression/expansion P-boxes, S-boxes with unequal input and output size). It combines **all the noninvertible elements in one unit**, the round function F, and uses the same unit in both encryption and decryption.

4.1 Structure and round equations

The block is divided into a **left half L** and a **right half R**. In every round the right half and the round key K_i are fed to F; the output of F is XORed with the left half (the *mixer*); and the two halves are then exchanged (the *swapper*). For round i = 1, 2, …, N:

$$L_i = R_{i-1}$$
$$R_i = L_{i-1} \oplus F(R_{i-1} , K_i)$$

Symbol	Meaning (beginner-friendly)
L_{i-1}, R_{i-1}	Left and right halves that **enter** round i. For the first round, L_0 and R_0 are the two halves of the plaintext.
L_i, R_i	Left and right halves that **leave** round i. After the last round, (L_N, R_N) is the ciphertext.
K_i	Round key of round i, produced by the round-key generator from the cipher key K.
F	Round function (S-boxes, P-boxes, etc.). It takes R_{i-1} and K_i and returns a value as wide as one half. It may be noninvertible.
⊕	Exclusive-or (mixer): bits that differ give 1, equal bits give 0.
$L_i = R_{i-1}$	This line is the **swap**: the old right half simply becomes the new left half. Only the new right half is changed.

Note: the classroom figure numbers the blocks L_1, R_1 → L_2, R_2 → L_3, R_3 and shows the mixer and swapper separately; here the plaintext halves are called L_0, R_0 so that round i produces L_i, R_i. The structure is identical.

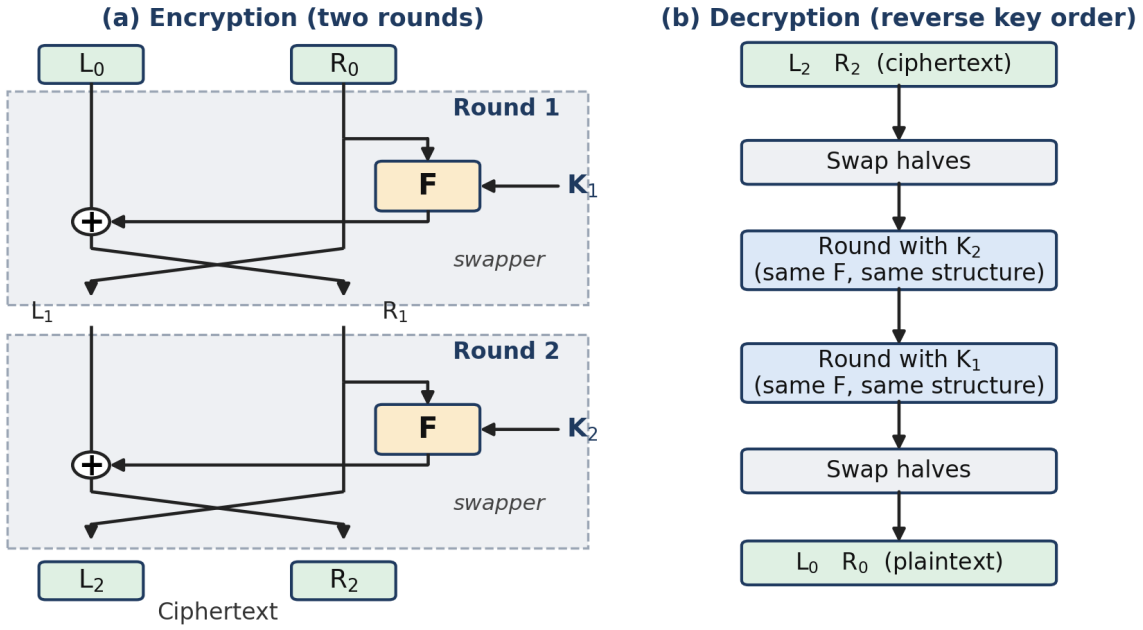


Figure 2. Two-round Feistel cipher: (a) encryption – each round XORs F(R, K_i) into the left half and swaps the halves; (b) decryption – the same round structure with the round keys in reverse order (K_2, K_1).

4.2 Why F does not need to be invertible

Decryption recovers the halves of round i from the output of round i:

$$R_{i-1} = L_i$$
$$L_{i-1} = R_i \oplus F(L_i, K_i)$$

Substituting the encryption equations, and using $L_i = R_{i-1}$:

$$R_i \oplus F(L_i, K_i) = L_{i-1} \oplus F(R_{i-1}, K_i) \oplus F(R_{i-1}, K_i) = L_{i-1} \oplus 00\ldots0 = L_{i-1}$$

F is always evaluated in the **forward** direction, with the same input (R_{i-1}) and the same key (K_i) in encryption and decryption, so its two outputs are identical and the XOR cancels them. Nobody ever has to compute F^{-1}; therefore F may contain compression/expansion P-boxes and noninvertible S-boxes (in DES the S-boxes turn 6 bits into 4). The only parts that must be invertible are the mixer (self-inverse XOR) and the swapper (self-inverse swap).

Design reasoning from the course: a first design, $C = P \oplus F(K)$, is reversible (see Section 5) but leaves the right half of the plaintext unchanged. The remedy is to **increase the number of rounds** and to **add a swapper** to every round, so that both halves are altered.

4.3 Decryption and the reverse order of round keys

A Feistel cipher uses the **same structure** for decryption: the ciphertext is processed through the same rounds, but the round keys are applied in **reverse order**. In the two-round example encryption uses $K_1 \rightarrow K_2$ and decryption uses $K_2 \rightarrow K_1$; in DES the 16 round keys are used as $K_1 \ldots K_{16}$ for encryption and $K_{16} \ldots K_1$ for decryption. Round i of decryption exactly undoes round N–i+1 of encryption, so after the final exchange of halves the plaintext (L_0, R_0) is restored.

4.4 DES as the Feistel example

DES splits its 64-bit block into two 32-bit halves and runs **16 Feistel rounds** with 16 round keys. Its round function F first passes R_{i-1} through an **expansion P-box (32 → 48 bits)**, XORs the result with the 48-bit round key, sends it through **eight S-boxes (6 → 4 bits each)** and finally through a **straight P-box (32 → 32 bits)**. The expansion P-box and the S-boxes are noninvertible, which is exactly why DES needs the Feistel structure to remain decryptable.

5. Worked XOR Example (from the Module 2 material)

The plaintext and ciphertext are each 4 bits and the key is 3 bits. The function takes the **first and third bits of the key**, reads them as a decimal number, **squares** it and writes the result as a 4-bit pattern. Encrypt and decrypt the plaintext **P = 0111** with the key **K = 101**, using $C = P \oplus F(K)$.

Step	Calculation	Explanation

1. Key-derived mask	`K = 1 0 1`
	`bits 1 and 3 → 11₂ = 3`
	`3² = 9 = 1001₂`
	`F(K) = 1001`
		The function extracts the first and third key bits, converts them to a decimal number, squares it and converts the result back to 4-bit binary. This 4-bit mask is the same in encryption and decryption.

2. Encryption	`  P    = 0111`
	`⊕ F(K) = 1001`
	`──────────`
	`  C    = 1110`
		XOR bit by bit: 0⊕1 = 1, 1⊕0 = 1, 1⊕0 = 1, 1⊕1 = 0. The ciphertext is 1110.

3. Decryption	`  C    = 1110`
	`⊕ F(K) = 1001`
	`──────────`
	`  P    = 0111`
		The same mask is XORed again: 1⊕1 = 0, 1⊕0 = 1, 1⊕0 = 1, 0⊕1 = 1. The original plaintext 0111 is recovered.

4. Why it works	`C ⊕ F(K)` `= P ⊕ F(K) ⊕ F(K) = P`	Because x ⊕ x = 0000, the mask cancels itself (P ⊕ 0000 = P); F(K) was never inverted.

Important: this is a **simplified XOR example, not a complete DES encryption**. It has one 4-bit block, no halves, no rounds and no S-boxes or P-boxes, whereas DES encrypts 64-bit blocks through 16 Feistel rounds. Its purpose is only to show how XOR cancels a function value that is never inverted. Here the mask depends only on the key; in a real Feistel round the mask is $F(R_{i-1}, K_i)$, which is why the right half passes through unchanged and the swapper and extra rounds are needed.

Additional illustration: one Feistel round with a noninvertible F

Take an 8-bit block with L_0 = 0111, R_0 = 1100 and the mask F(K) = 1001 from above. Let the (deliberately noninvertible) round function be F(R, K) = (R AND 1010) ⊕ F(K); the AND discards information, so F cannot be inverted.

Operation	Calculation	Result
Encryption: $F(R_0, K_1)$	`(1100 AND 1010) ⊕ 1001 = 1000 ⊕ 1001`	0001
$R_1 = L_0 \oplus F$	`0111 ⊕ 0001`	R_1 = 0110
$L_1 = R_0$	`old right half becomes new left half`	L_1 = 1100
Decryption: $R_0 = L_1$	`L₁ = 1100`	R_0 = 1100
$L_0 = R_1 \oplus F(L_1, K_1)$	`0110 ⊕ [(1100 AND 1010) ⊕ 1001]` `= 0110 ⊕ 0001`	L_0 = 0111 ✓

The original left half is recovered even though F was never inverted – the same value $F(R_0, K_1)$ = 0001 was simply XORed in twice.

6. Non-Feistel Ciphers

A **non-Feistel cipher** uses **only invertible components**. Each component in the encryption cipher has a corresponding component in the decryption cipher, and the whole block is processed together – there is **no need to divide the plaintext into two halves**. Each round applies a key mixer, S-boxes and a P-box (Figure 3); with X_0 the plaintext and X_N the ciphertext:

$$\text{Encryption: } X_i = P(\,S(\,X_{i-1} \oplus K_i\,)\,)$$
$$\text{Decryption: } X_{i-1} = S^{-1}(\,P^{-1}(\,X_i\,)\,) \oplus K_i$$

Restrictions on the components (as in the course material): (1) S-boxes must have an **equal number of inputs and outputs** and be one-to-one so that a decryption S-box exists; (2) **no compression or expansion P-boxes** are allowed, only straight P-boxes, which have inverse permutation tables; (3) the XOR key mixer is self-invertible; (4) the plaintext need not be split into two halves. Decryption therefore applies the **inverse of each component in the reverse order**, with the round keys also in reverse order, so the decryption algorithm is *different* from the encryption algorithm.

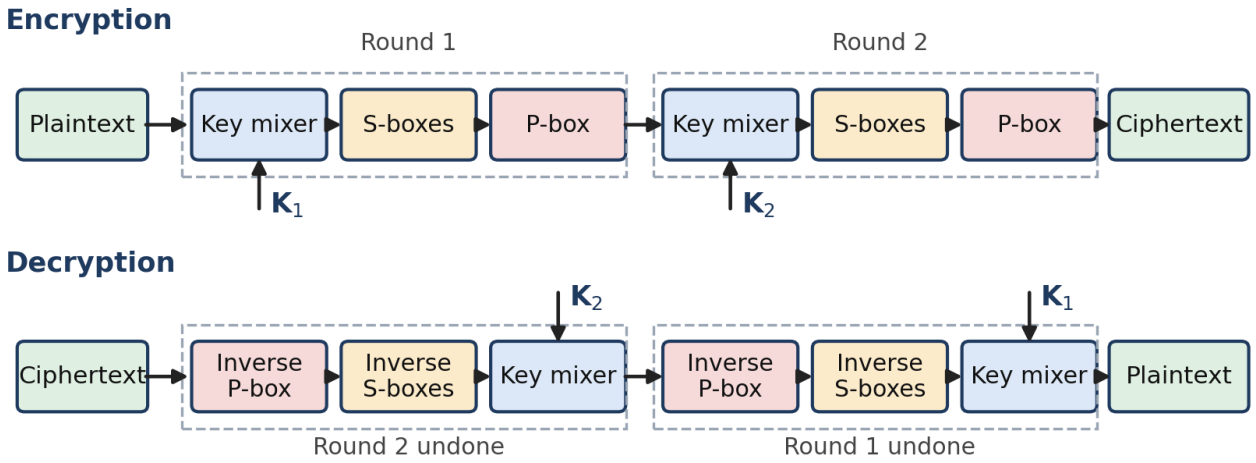


Figure 3. Non-Feistel cipher (two rounds, 8-bit block): encryption applies key mixer → S-boxes → P-box; decryption applies inverse P-box → inverse S-boxes → key mixer with the round keys in reverse order.

Example – AES. AES processes a 128-bit block for 10, 12 or 14 rounds. A round uses SubBytes (S-box substitution), ShiftRows (permutation), MixColumns (diffusion) and AddRoundKey (XOR with the round key). Each step has an inverse (InvSubBytes, InvShiftRows, InvMixColumns; AddRoundKey is self-inverse), and decryption applies them in reverse order. AES is not divided into Feistel halves.

7. Feistel vs Non-Feistel Ciphers

Feature	Feistel cipher	Non-Feistel cipher
Data arrangement	Block split into left and right halves; only one half is modified per round and the halves are swapped.	Whole block processed together; no division into halves.
Component invertibility	May use self-invertible, invertible and noninvertible components; the noninvertible ones are grouped in F.	Only invertible components: equal-size S-boxes, straight P-boxes, XOR.
Round structure	$L_i = R_{i-1}$ $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$	$X_i = P(S(X_{i-1} \oplus K_i))$
Encryption / decryption	Same structure; only the round keys are used in reverse order.	Different algorithm: inverse components applied in reverse order.
Round keys	K_i feeds F in round i; reversed for decryption.	K_i feeds the key mixer in round i; reversed for decryption.
Examples	DES (16 rounds)	AES

8. Security Role of Product Ciphers

- **Key mixing (XOR)** ties every round to a round key, so the data cannot be analysed without the key – it supports **confusion**.
- **Substitution (S-boxes)** supplies the nonlinearity; without it the cipher would be a simple linear relation between plaintext, key and ciphertext. It is the main source of **confusion**.
- **Permutation (P-boxes)** moves the output bits of each S-box into different S-boxes of the next round, so a local change spreads – the main source of **diffusion**.
- **Repeated rounds** let this combined effect accumulate: the ciphertext looks more and more like a random word, and more rounds also involve more round keys, which better hides the key–ciphertext relationship. In a Feistel cipher each round changes only one half, so at least two rounds are needed before both halves are affected; practical ciphers use larger blocks, more S-boxes and more rounds (DES: 16).

Conclusion

Product ciphers build strong block ciphers by combining key mixing, substitution and permutation in repeated rounds, thereby providing diffusion and confusion. They fall into two classes. Feistel ciphers, such as DES, split the block into halves and obey $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$; the XOR mixer allows the noninvertible round function to cancel itself, so decryption reuses the same structure with the round keys reversed. Non-Feistel ciphers, such as AES, restrict themselves to invertible components and decrypt by applying the inverse of every component in the opposite order.

Key Points to Remember

- Product cipher = substitution + permutation + key mixing, iterated over rounds; **diffusion** hides plaintext–ciphertext relations, **confusion** hides key–ciphertext relations.
- Feistel: halves L and R, round function F, round key K_i, XOR, swap; F need **not** be invertible. Non-Feistel: **all** components invertible, **no** halves.
- Feistel decryption = same structure with keys $K_N \ldots K_1$; non-Feistel decryption = inverse components in reverse order.
- In the worked example, F(K) = 1001, 0111 ⊕ 1001 = 1110 and 1110 ⊕ 1001 = 0111; it is only a simplified illustration, not DES.
- Non-Feistel restrictions: equal-size S-boxes and no compression or expansion P-boxes. Examples: DES (Feistel), AES (non-Feistel).

Data Encryption Standard (DES)

Contents

1	Introduction and background
2	Overall structure of DES
3	Initial and final permutations
4	Rounds: mixer and swapper
5	The DES function (expansion, XOR, S-boxes, straight P-box)
6	Cipher and reverse cipher (two approaches) and pseudocode
7	Key generation (round-key generator)
8	Worked example from the slides
9	Quick revision sheet

1. Introduction and Background

The **Data Encryption Standard (DES)** is a **symmetric-key block cipher** published by the **National Institute of Standards and Technology (NIST)**.

1.1 Where DES fits: product ciphers

Modern block ciphers are all **product ciphers**, and they are divided into two classes:

- **Feistel ciphers** – e.g., **DES**
- **Non-Feistel ciphers** – e.g., AES

So DES is a Feistel cipher: every round of DES is itself a Feistel cipher.

1.2 History

- In **1973**, NIST published a request for proposals for a national symmetric-key cryptosystem.
- A proposal from **IBM**, a modification of a project called **Lucifer**, was accepted as DES.
- DES was published in the Federal Register in **March 1975** as a draft of the Federal Information Processing Standard (FIPS).
- The draft was criticised severely for two reasons: (1) the **small key length (only 56 bits)**, which could make the cipher vulnerable to brute-force attack; (2) concern about **hidden design** in the internal structure of DES. IBM designers later stated that the internal structure was designed to prevent **differential cryptanalysis**.
- DES was finally published as **FIPS 46** in the Federal Register in **January 1977**.

2. Overall Structure of DES

DES is a block cipher: it encrypts a **64-bit plaintext block** into a **64-bit ciphertext block** using a cipher key. The encryption process is made of **two permutations (P-boxes)**, called the **initial permutation** and the **final permutation**, and **sixteen Feistel rounds**. Each round uses a **different 48-bit round key** generated from the cipher key by the **round-key generator**.

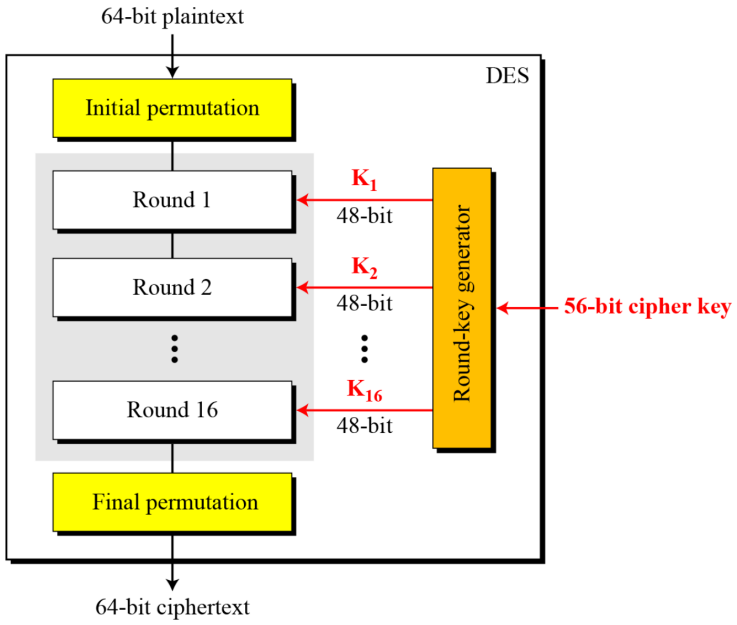


Figure 1: General structure of DES (slide 8)

Flow of data: 64-bit plaintext → Initial permutation → Round 1 → Round 2 → … → Round 16 → Final permutation → 64-bit ciphertext. The round-key generator feeds K_1, K_2, …, K_{16} (48 bits each) to the 16 rounds, taking a 56-bit cipher key as input.

> **Key numbers:** block = 64 bits | effective cipher key = 56 bits (64 with parity) | rounds = 16 | round key = 48 bits

3. Initial and Final Permutations

Each of these permutations takes a **64-bit input** and permutes the bits according to a **predefined rule**. They are **keyless straight permutations** and they are the **inverse of each other**.

Example: in the initial permutation, the **58th bit** of the input becomes the **first bit** of the output.

No cryptographic significance: both permutations are keyless and predetermined. The guess is that DES was designed to be implemented in **hardware (on chips)**.

Initial permutation

58	50	42	34	26	18	10	02
60	52	44	36	28	20	12	04
62	54	46	38	30	22	14	06
64	56	48	40	32	24	16	08
57	49	41	33	25	17	09	01
59	51	43	35	27	19	11	03
61	53	45	37	29	21	13	05
63	55	47	39	31	23	15	07

Final permutation

40	08	48	16	56	24	64	32
39	07	47	15	55	23	63	31
38	06	46	14	54	22	62	30
37	05	45	13	53	21	61	29
36	04	44	12	52	20	60	28
35	03	43	11	51	19	59	27
34	02	42	10	50	18	58	26
33	01	41	09	49	17	57	25

Table 1: Initial and final permutation tables (slide 10). Read row by row: the i-th number tells which input bit goes to output position i.

Remember: the initial and final permutations are straight P-boxes that are inverses of each other. They have no cryptographic significance in DES.

4. Rounds: Mixer and Swapper

DES uses **16 rounds**, and **each round of DES is a Feistel cipher**. A round takes L_{I-1} and R_{I-1} (32 bits each) from the previous round and creates L_I and R_I, which go to the next round.

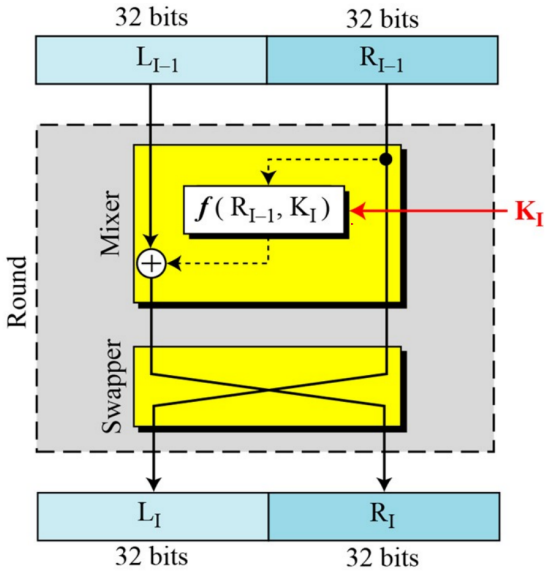


Figure 2: One round of DES – mixer and swapper (slide 12)

Each round has **two cipher elements**:

- **Mixer** – combines the left half with the output of $f(R_{I-1}, K_I)$ using **XOR**. The mixer is **invertible because of the XOR operation**.
- **Swapper** – swaps the left half of the text with the right half. It is obviously invertible.

Each of these elements is invertible. **All non-invertible elements are collected inside the function $f(R_{I-1}, K_I)$**.

5. The DES Function

The **heart of DES** is the DES function. It applies a **48-bit key** to the rightmost **32 bits (R_{I-1})** to produce a **32-bit output**. It is made up of four sections: an **expansion P-box**, a **whitener** (that adds the key), a group of **S-boxes**, and a **straight P-box**.

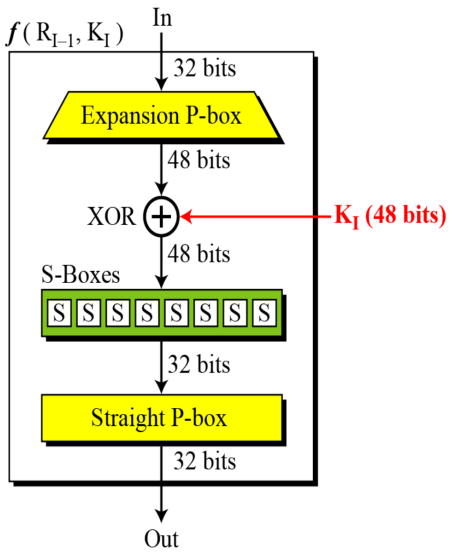


Figure 3: The DES function f(R_{I-1}, K_I) (slide 13)

5.1 Expansion P-box (32 → 48 bits)

R_{I-1} is a 32-bit input while K_I is a 48-bit key, so R_{I-1} must first be expanded to 48 bits. R_{I-1} is divided into **eight 4-bit sections**, and each 4-bit section is expanded to **6 bits** by a predetermined rule:

- Input bits **1, 2, 3, 4** of a section are copied to output bits **2, 3, 4, 5** respectively.
- **Output bit 1** comes from bit 4 of the **previous** section.
- **Output bit 6** comes from bit 1 of the **next** section.
- Sections 1 and 8 are considered adjacent, so the same rule applies to bits 1 and 32.

Although the relationship can be defined mathematically, DES uses a table (each row below gives the input bits for one 4-bit section's six output bits; the shaded first and last columns are the borrowed neighbour bits).

32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	01

Table 2: Expansion permutation table (slide 16)

5.2 Whitener (XOR)

After the expansion permutation, DES performs an **XOR** of the expanded right section (48 bits) with the **round key** (48 bits). Both are 48 bits long. **The round key is used only in this operation.**

5.3 S-boxes (48 → 32 bits)

The S-boxes do the **real mixing (confusion)**. DES uses **8 S-boxes**, each with a **6-bit input** and a **4-bit output**. The 48-bit data from the XOR step is divided into **eight 6-bit chunks**, each chunk is fed into one S-box, and each box gives a 4-bit chunk. Combined, the eight outputs form a **32-bit** text.

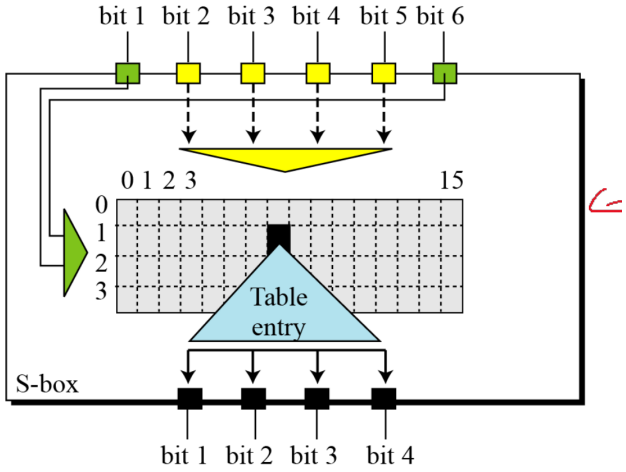


Figure 4: S-box rule (slide 19)

S-box rule. Each S-box follows a predetermined rule based on a **4-row by 16-column table**:

- The combination of **bits 1 and 6** of the input defines one of the **four rows**.
- The combination of **bits 2 through 5** defines one of the **sixteen columns**.
- Each S-box has its own table, so **eight tables** are needed. Row/column numbers and outputs are given in decimal in the tables to save space; they must be changed to binary.

S-box 1 (slide 21) – the other seven boxes are in the textbook:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

Table 3: S-box 1 (values in decimal)

Example 1. The input to S-box 1 is `100011`. What is the output?

- First and sixth bits together: `11` = 3 (row 3).
- Remaining bits: `0001` = 1 (column 1).
- S-box 1, row 3, column 1 = **12** = `1100`.

So the input 100011 yields the output 1100.

Example 2. The input to S-box 8 is `000000`. What is the output?

- First and sixth bits: `00` = 0 (row 0); remaining bits `0000` = 0 (column 0).
- S-box 8, row 0, column 0 = **13** = `1101`.

So the input 000000 yields the output 1101.

5.4 Straight permutation (32 → 32 bits)

The 32-bit output of the S-boxes goes through a **straight permutation** (a straight P-box) using the table below. The result is the 32-bit output of the DES function.

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Table 4: Straight permutation table (slide 25)

DES function in one line: 32-bit R_{I-1} → expansion (48) → XOR with 48-bit K_I → 8 S-boxes (6→4 each, 32 in total) → straight P-box (32).

6. Cipher and Reverse Cipher

Using mixers and swappers, we can create the **cipher** and the **reverse cipher**, each having **16 rounds**. There are two ways to arrange the rounds.

6.1 First approach

Make the **last round (round 16) different** from the others: it has **only a mixer and no swapper**.

Note: In the first approach, there is no swapper in the last round.

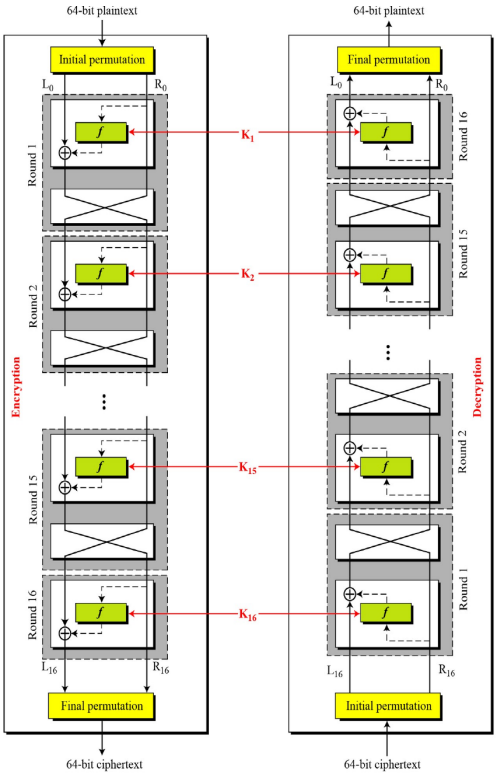


Figure 5: DES cipher (left) and reverse cipher (right) for the first approach (slide 27)

In the figure, the encryption side runs Initial permutation → Round 1 (with K_1) → … → Round 16 (with K_{16}) → Final permutation. The decryption side is the mirror image: Final permutation → Round 16 → … → Round 1 → Initial permutation, using the round keys in the reverse order (K_{16} first, K_1 last).

6.2 Alternative approach

Make **all 16 rounds the same** by including one swapper in the 16th round, and then add an **extra swapper after that**. The two swappers cancel the effect of each other.

6.3 Pseudocode for the DES cipher (Algorithm 6.1)

Main cipher (slide 28):

```
Cipher (plainBlock[64], RoundKeys[16, 48], cipherBlock[64])
{
    permute (64, 64, plainBlock, inBlock, InitialPermutationTable)
    split   (64, 32, inBlock, leftBlock, rightBlock)
    for (round = 1 to 16)
    {
        mixer (leftBlock, rightBlock, RoundKeys[round])
        if (round != 16) swapper (leftBlock, rightBlock)
    }
    combine (32, 64, leftBlock, rightBlock, outBlock)
    permute (64, 64, outBlock, cipherBlock, FinalPermutationTable)
}
```

Mixer and swapper (slide 29):

```
mixer (leftBlock[48], rightBlock[48], RoundKey[48])
{
    copy (32, rightBlock, T1)
    function (T1, RoundKey, T2)
    exclusiveOr (32, leftBlock, T2, T3)
    copy (32, T3, rightBlock)
}

swapper (leftBlock[32], rightBlock[32])
{
    copy (32, leftBlock, T)
    copy (32, rightBlock, leftBlock)
    copy (32, T, rightBlock)
}
```

DES function and substitution (slides 30–31):

```
function (inBlock[32], RoundKey[48], outBlock[32])
{
    permute     (32, 48, inBlock, T1, ExpansionPermutationTable)
    exclusiveOr (48, T1, RoundKey, T2)
    substitute  (T2, T3, SubstitutionTables)
    permute     (32, 32, T3, outBlock, StraightPermutationTable)
}

substitute (inBlock[32], outBlock[48], SubstitutionTables[8, 4, 16])
{
    for (i = 1 to 8)
    {
        row = 2 x inBlock[i x 6 + 1] + inBlock[i x 6 + 6]
        col = 8 x inBlock[i x 6 + 2] + 4 x inBlock[i x 6 + 3] +
              2 x inBlock[i x 6 + 4] + inBlock[i x 6 + 5]
        value = SubstitutionTables[i][row][col]
        outBlock[i x 4 + 1] = value / 8;   value = value mod 8
        outBlock[i x 4 + 2] = value / 4;   value = value mod 4
        outBlock[i x 4 + 3] = value / 2;   value = value mod 2
        outBlock[i x 4 + 4] = value
    }
}
```

Reading the pseudocode: the plaintext goes through the initial permutation, is split into two 32-bit halves, then goes through 16 rounds of mixer; a swapper is applied after every round *except round 16* (first approach). The halves are combined and the final permutation gives the ciphertext. Inside the function, the row is formed from the 1st and 6th bits and the column from bits 2–5 of every 6-bit chunk, matching the S-box rule.

7. Key Generation

The **round-key generator** creates **sixteen 48-bit keys** out of a **56-bit cipher key**. However, the cipher key is normally a **64-bit key** in which **8 extra bits are parity bits** (bits **8, 16, 24, 32, 40, 48, 56, 64**).

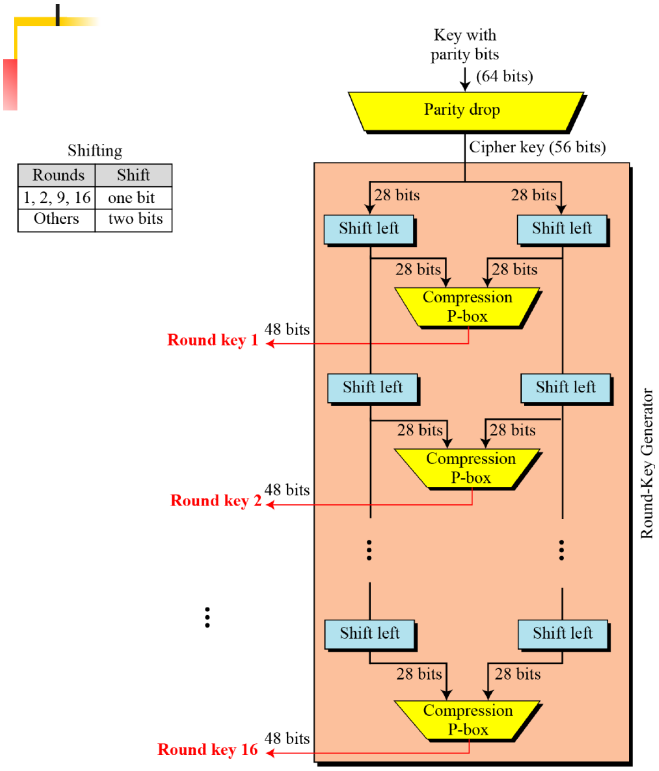


Figure 6: Key generation (slide 34)

Steps

- **Parity drop:** the 64-bit key passes through the parity-drop permutation, giving the **56-bit cipher key**.
- **Split:** the 56 bits are divided into two **28-bit** halves (left key and right key).
- **Shift left:** each 28-bit half is shifted left by **one or two bits**, according to the round (see Table 6).
- **Combine:** the two halves are joined again (28 + 28 = 56 bits).
- **Compression P-box:** the compression permutation changes **56 bits to 48 bits**, giving the round key for that round.
- The process is repeated for rounds 1 to 16 to obtain K_1 … K_{16}. The shifted halves carry over from one round to the next.

> **Note on the slide:** slide 37 says the compression permutation changes “58 bits to 48 bits”. The key at that stage is 56 bits (see the figure and the pseudocode *permute (56, 48, ...)*), so read it as **56** → **48**.

Parity-drop table (64 → 56), slide 35:

57	49	41	33	25	17	09	01
58	50	42	34	26	18	10	02
59	51	43	35	27	19	11	03
60	52	44	36	63	55	47	39
31	23	15	07	62	54	46	38
30	22	14	06	61	53	45	37
29	21	13	05	28	20	12	04

Table 5: Parity-bit drop table

Number of bit shifts per round (slide 36): rounds 1, 2, 9 and 16 shift by **one bit**; all other rounds shift by **two bits**.

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bit shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Table 6: Number of bit shifts

Key-compression table (56 → 48), slide 37:

14	17	11	24	01	05	03	28
15	06	21	10	23	19	12	04
26	08	16	07	27	20	13	02
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

Table 7: Key-compression table

Algorithm for round-key generation (slides 38–39):

```
Key_Generator (keyWithParities[64], RoundKeys[16, 48], ShiftTable[16])
{
    permute (64, 56, keyWithParities, cipherKey, ParityDropTable)
    split   (56, 28, cipherKey, leftKey, rightKey)
    for (round = 1 to 16)
    {
        shiftLeft (leftKey,  ShiftTable[round])
        shiftLeft (rightKey, ShiftTable[round])
        combine (28, 56, leftKey, rightKey, preRoundKey)
        permute (56, 48, preRoundKey, RoundKeys[round], KeyCompressionTable)
    }
}

shiftLeft (block[28], numOfShifts)
{
    for (i = 1 to numOfShifts)
    {
        T = block[1]
        for (j = 2 to 28)  block[j-1] = block[j]
        block[28] = T
    }
}
```

8. Worked Example from the Slides

A random plaintext block and a random key are chosen (all in hexadecimal), and the ciphertext is determined (slides 40–42):

Plaintext	123456ABCD132536
Key	AABB09182736CCDD
Ciphertext	C0B7A8D05F3A829C

After the initial permutation: `14A7D67818CA18AD`. After splitting: L_0 = `14A7D678`, R_0 = `18CA18AD`.

Round	Left	Right	Round key
Round 1	18CA18AD	5A78E394	194CD072DE8C
Round 2	5A78E394	4A1210F6	4568581ABCCE
Round 3	4A1210F6	B8089591	06EDA4ACF5B5
Round 4	B8089591	236779C2	DA2D032B6EE3
Round 5	236779C2	A15A4B87	69A629FEC913
Round 6	A15A4B87	2E8F9C65	C1948E87475E
Round 7	2E8F9C65	A9FC20A3	708AD2DDB3C0
Round 8	A9FC20A3	308BEE97	34F822F0C66D
Round 9	308BEE97	10AF9D37	84BB4473DCCC
Round 10	10AF9D37	6CA6CB20	02765708B5BF
Round 11	6CA6CB20	FF3C485F	6D5560AF7CA5
Round 12	FF3C485F	22A5963B	C2C1E96A4BF3
Round 13	22A5963B	387CCDAA	99C31397C91F
Round 14	387CCDAA	BD2DD2AB	251B8BC717D0
Round 15	BD2DD2AB	CF26B472	3330C5D9A36D
Round 16	19BA9212	CF26B472	181C5D75C66D

Table 8: Trace of data for the example (slides 40–42)

After combination: `19BA9212CF26B472`. After the final permutation the ciphertext is `C0B7A8D05F3A829C`.

Observations from the trace:

- From round 1 to round 15, each round's Left equals the previous round's Right (the swapper at work), e.g. $L_1 = R_0 = 18CA18AD$.
- In round 16 there is **no swap** (first approach): the Right half stays `CF26B472` as in round 15.

Decryption of the same block (slide 43)

Bob deciphers the ciphertext `C0B7A8D05F3A829C` with the **same key**. After the initial permutation the text is `19BA9212CF26B472`; after splitting, L_0 = `19BA9212`, R_0 = `CF26B472`. Round 1 of decryption uses the round key `181C5D75C66D` (which was K_{16} of encryption), round 2 uses `3330C5D9A36D` (K_{15}), … and the last round uses `194CD072DE8C` (K_1). After combination the text is `14A7D67818CA18AD`, and the final permutation gives back the plaintext `123456ABCD132536`.

This confirms that decryption is the **same algorithm with the round keys in reverse order**.

9. Quick Revision Sheet

Item	Value / fact
Type	Symmetric-key block cipher; Feistel cipher; published by NIST
Block size	64 bits plaintext and ciphertext
Cipher key	56 effective bits (64 bits including 8 parity bits)
Rounds	16 Feistel rounds, each with a mixer and a swapper (no swapper in round 16, first approach)
Round key	48 bits, K1 ... K16, from the round-key generator
Initial / final permutation	Keyless, straight P-boxes, inverses of each other, no cryptographic significance
Halves	L and R, 32 bits each
DES function	Expansion P-box (32 to 48) > XOR with round key > 8 S-boxes (48 to 32) > straight P-box (32 to 32)
S-box	6-bit input, 4-bit output; rows from bits 1 and 6, column from bits 2-5; 4 x 16 table; 8 tables
Key generation	Parity drop (64 to 56) > split 28+28 > shift left 1 or 2 > combine > compress (56 to 48)
Shifts	1 bit in rounds 1, 2, 9, 16; 2 bits in the others
Parity bits	8, 16, 24, 32, 40, 48, 56, 64
Decryption	Same structure, round keys used in reverse order

Practice questions (answers are in these notes)

1. Draw and explain the general structure of DES.
2. Why do the initial and final permutations have no cryptographic significance?
3. Explain the two elements of a DES round (mixer and swapper). Why is each invertible?
4. Describe the four sections of the DES function with the bit sizes at each stage.
5. Explain how the expansion P-box expands a 32-bit input to 48 bits.
6. How is the S-box row and column selected? Find the output of S-box 1 for the input 100011.
7. Explain the two approaches to building the DES cipher and reverse cipher.
8. Explain the DES round-key generation process with a diagram. How many bits are shifted in each round?

Jayasham ND
4MC23CS059

DES Analysis

Data Encryption Standard (DES) · Chapter 6, Section 6.3 · Properties, design criteria (S-boxes, P-boxes, rounds)

1. Overview: what is "DES Analysis"?

Critics have examined DES very closely. Researchers tested whether it has the properties a good block cipher should have, checked whether its components met the designers' stated criteria, and searched for weaknesses.

The section breaks down as:

Part	What it covers
Properties	Avalanche effect, completeness effect
Design criteria	S-boxes, P-boxes, number of rounds
Weaknesses in cipher design	S-box and P-box weaknesses
Weaknesses in the cipher key	Key size, weak keys, semi-weak keys, key complement, key clustering

2. Desired Properties of a Block Cipher

2.1 Avalanche effect

Definition: A small change in the plaintext (or key) should produce a significant change in the ciphertext.

Why it matters: If flipping one input bit changed only one output bit, an attacker could spot patterns and work backwards. A strong avalanche makes the input–output relationship unpredictable.

Textbook Example 6.7: Encrypt two plaintext blocks with the *same key* that differ in only the rightmost bit:

	Plaintext	Key	Ciphertext
Block 1	0000000000000000	22234512987ABB23	4789FD476E82A5F1
Block 2	0000000000000001	22234512987ABB23	0A4ED5C15A63FEA3

- The ciphertexts differ in **29 bits** out of 64, which is 29/64 × 100 ≈ **45%**.

2.2 Completeness effect

Definition: Each bit of the ciphertext should depend on *many* bits of the plaintext.

DES achieves this through the **diffusion and confusion** created by its P-boxes and S-boxes, so DES shows a very strong completeness effect. Think of it as influence spreading: one plaintext bit affects several bits after one round, and still more after repeated rounds.

2.3 Avalanche vs completeness

Avalanche effect	Completeness effect
Small input change → large output change	Each output bit depends on many input bits
Focuses on the *amount of change*	Focuses on the *dependency between bits*
Example: 1 changed input bit → 29 output bits change	Example: a ciphertext bit is influenced by many plaintext bits

3. Design Criteria of DES

DES's design was revealed by IBM in **1994**. Many tests have shown it satisfies most of the claimed criteria. The textbook covers three areas: **S-boxes, P-boxes and number of rounds**.

3.1 S-box design criteria

Each DES S-box takes **6 input bits** and gives **4 output bits**; there are **8 S-boxes** in the round function. For example, input 101011 is looked up in the S-box table and gives a 4-bit output (illustrative only, not a verified lookup).

All eight rules in one table

Rule	Simple meaning	Main purpose
1	Each row contains 0–15 exactly once.	Ensures each row is a permutation.
2	The input-output relationship is nonlinear.	Prevents simple mathematical prediction.
3	Change one input bit → at least two output bits change.	Ensures a strong response to a one-bit change.
4	Flip middle bits 3 and 4 → at least two output bits change.	Controls specific input differences.
5	Flip the first two bits (last two unchanged, middle bits arbitrary) → output must differ.	Prevents specific input pairs from having equal outputs.
6	For the specified 32 pairs, no output difference occurs more than eight times.	Limits repeated difference patterns.
7	Apply a similar difference criterion to three S-boxes.	Checks patterns across combinations of S-boxes.
8	Minimise imbalance between zeros and ones under the specified input conditions.	Reduces statistical bias.

Rule 1 – Every row contains 0 to 15 exactly once

- **Rule:** Each row of an S-box is a permutation of the 16 four-bit values (0 to 15). The order can vary, but nothing repeats and nothing is missing.
- **Example:** A valid row is 14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 5, 9, 0, 7, 12, since all 16 values appear once. The row 14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 5, 9, 0, 7, 7 is invalid: 7 repeats and 12 is missing.

Rule 2 – S-boxes must be nonlinear

- **Rule:** The output must not be an affine (simple linear) transformation of the input, so no easy formula predicts it.
- **Example:** Suppose a hypothetical substitution works like y = x ⊕ 0011, where ⊕ means XOR.

If the input is 1010:

```
  1010
⊕ 0011
------
  1001
```

If the input changes to 1011:

```
  1011
⊕ 0011
------
  1000
```

The relationship is easy to describe: simply XOR the input with a fixed value. This illustrates a simple affine relationship, **not** a valid DES S-box (it uses four-bit inputs only to keep the calculation easy).

A nonlinear substitution does not follow such a simple linear or affine rule. A real S-box produces outputs with no single fixed XOR mask or simple linear equation explaining the input–output relationship.

Rule 3 – Change 1 input bit → at least 2 output bits change

- **Rule:** If two 6-bit inputs differ in exactly one bit, their 4-bit outputs must differ in at least two bits.

Step 1: Take two inputs. Original input 101101; change only the last bit to get 101100.

```
101101
101100
     ↑
Only one input bit changed
```

Step 2: Apply the S-box. For illustration, suppose the outputs are:

```
Input 1: 101101 → 1101
Input 2: 101100 → 0011
```

These values only illustrate the required difference; they are not actual outputs of a particular DES S-box.

Step 3: Compare the outputs.

```
Output 1: 1101
Output 2: 0011
           ↑↑
```

Bit position	Output 1	Output 2	Changed?
1	1	0	Yes
2	1	0	Yes
3	0	1	Yes
4	1	1	No

Three output bits changed, which satisfies the rule (at least two must change).

Why important? It prevents a one-bit input change from causing only a one-bit output change, giving a stronger response.

Remember: One input bit changes → at least two output bits must change.

Rule 4 – Flip the two middle bits (3 and 4) → at least 2 output bits change

- **Rule:** S(x) and S(x ⊕ 001100) must differ in at least two bits.

Step 1: Understand the bit positions.

```
Bit position:  1 2 3 4 5 6
Original:      1 0 1 1 0 1
                   ↑ ↑
                 Middle bits
```

Step 2: Change bits 3 and 4. The two middle bits are 11; complement both to get 00.

```
Original: 101101
New:      100001
            ↑↑
```

The other four bits remain unchanged. The same operation is $x' = x \oplus 001100$ (XOR with 001100 flips bits 3 and 4).

Step 3: Compare the outputs. Suppose the illustrative outputs are:

```
Input 1 → 1011
Input 2 → 0100

1011
0100
----
↑↑↑↑
```

All four positions differ, so the outputs differ in at least two positions.

Why important? It prevents a specific two-bit input change from producing outputs that differ in only zero or one position.

Remember: Flip the two middle bits → at least two output bits must change.

Rule 5 – Change the first two bits → the output must differ

- **Rule:** If two inputs differ in the first two bits (1, 2) and are the same in the last two bits (5, 6), their outputs must be different. In symbols, $S(x) \neq S(x \oplus 11bc00)$, where b and c can be anything.

Step 1: Start with an input. The six-bit input is 101101.

```
Original: 10 | 1101
          ↑↑
         First two bits
```

Step 2: Complement the first two bits. Changing 10 to 01 gives:

```
Original: 10 | 1101
New:      01 | 1101
```

The last four bits remain unchanged.

Step 3: Compare the outputs. Suppose the illustrative outputs are:

```
Original input → 1010
New input      → 0111
```

These outputs are different, as required. The textbook expresses the rule as $S(x) \neq S(x \oplus 11bc00)$, where b and c can each be 0 or 1. In 11bc00, the first two bits are flipped, the last two bits are unchanged, and the two middle positions depend on b and c. The condition ensures the two outputs can never be equal.

Remember: Flip the first two bits, keep the last two, middle bits arbitrary → the output must change.

Rule 6 – No output difference may repeat more than 8 times

- **Rule:** Take the 32 input pairs (x_i, x_j) with $x_i \oplus x_j \neq 000000$. Compute the output difference $d = y_i \oplus y_j$ for each pair. No single value of d may occur more than 8 times.

Step 1: Take two inputs. For illustration (assume the two inputs differ by a nonzero six-bit value):

```
Input X₁ → S-box → Y₁ = 1100
Input X₂ → S-box → Y₂ = 0110
```

Step 2: XOR the outputs. XOR gives 1 when the two bits differ and 0 when they are the same.

```
   1100
⊕  0110
--------
   1010
```

Therefore $d = Y_1 \oplus Y_2 = 1010$, a four-bit value that represents the difference between the outputs.

Step 3: Repeat for 32 selected input pairs. The textbook considers 32 non-identical input pairs and calculates the output XOR difference for each. Imagine the results are:

Input pair	Output difference
Pair 1	1010
Pair 2	0110
Pair 3	1101
Pair 4	0011
Pair 5	1010
...	...
Pair 32	0101

The example above is illustrative, not the actual DES S-box data.

Step 4: Count repeated differences. Suppose 1010 occurs six times. That is acceptable under the textbook limit: no output difference should occur more than eight times among the 32 selected comparisons.

Why important? If the same output difference occurred too often, it could reveal a predictable relationship between input and output differences. Limiting repetition helps resist differential cryptanalysis.

Rule 7 – A similar rule applies to three S-boxes

- **Rule:** The idea of Rule 6 (limit repeated output differences) is extended to combinations of three S-boxes.

Simple example. Imagine information passing through three substitution stages:

```
Input
  |
  v
[S-box 1]
  |
  v
[S-box 2]
  |
  v
[S-box 3]
  |
  v
Output
```

An attacker might try to find input changes that consistently produce particular output changes. If several S-boxes together create repeated or predictable difference patterns, those patterns could help the attacker analyse the cipher. The designers therefore apply a similar difference-based criterion to combinations involving three S-boxes.

Why important? A substitution pattern should not become easy to exploit merely because several S-boxes are considered together.

Important textbook limitation: The text only says that a criterion similar to Rule 6 applies to three S-boxes. It does not give the exact numerical condition for Rule 7, so we should not invent one.

Rule 8 – Keep the number of 0s and 1s balanced

- **Rule:** If one input bit is held fixed (0 or 1) while the other bits vary, the output should have nearly equal numbers of 0s and 1s.

Step 1: Fix one input bit. For example, keep the first bit at 0 and let the remaining five bits vary:

```
0 _ _ _ _ _

Possible inputs:
000000
000001
000010
000011
...
011111
```

There are 32 possible combinations when five bits vary.

Step 2: Apply the S-box. Each input produces a four-bit output. Across the 32 inputs there are 128 output-bit positions in total.

Step 3: Count the zeros and ones. Imagine the output bits are distributed like this (approximately balanced):

Bit value	Example count
Number of 0s	63
Number of 1s	65
Total	128

Now imagine a very unbalanced result (heavily biased toward zeros):

Bit value	Example count
Number of 0s	110
Number of 1s	18
Total	128

These numbers are only illustrative; they are not measured values from a particular DES S-box.

> **Remember:** The S-box criteria aim to give **nonlinearity, confusion and strong bit changes**, making DES harder to attack.

3.2 P-box design criteria

A P-box only **rearranges bit positions**; it does not change bit values (e.g. 101100 → 110010: same count of 0s and 1s, new positions).

Between two rows of S-boxes (two consecutive rounds) there are:

- one **straight P-box** (32 → 32), and
- one **expansion P-box** (32 → 48).

Together they provide **diffusion** of bits.

The 7 design criteria:

1. Each S-box input comes from the output of a *different* S-box in the previous round.
2. No input to a given S-box comes from the output of the *same* box in the previous round.
3. The four outputs from each S-box go to **six different** S-boxes in the next round.
4. No two output bits from an S-box go to the *same* S-box in the next round.
5. Number the S-boxes $S_1 \ldots S_8$. Then: (a) an output of S_{j-2} goes to one of the **first two bits** of S_j; (b) an output bit of S_{j-1} goes to one of the **last two bits** of S_j; (c) an output of S_{j+1} goes to one of the **two middle bits** of S_j.

6 For each S-box, **two** output bits go to the first or last two bits of an S-box in the next round, and the **other two** go to middle bits of an S-box in the next round.

7 If an output bit from S_j goes to a middle bit of S_k in the next round, then an output bit from S_k cannot go to a middle bit of S_j. Setting j = k means no S-box output can go to its own middle input bits in the next round.

Why? If all bits from one S-box stayed together, their influence would stay local. Spreading them across different S-boxes lets a change propagate quickly through the whole cipher.

> **Exam point:** S-boxes = substitution (confusion). P-boxes = permutation (diffusion). Together they build strong DES rounds.

3.3 Number of rounds

- DES uses **16 rounds** of Feistel cipher.
- It has been proved that after **8 rounds**, every ciphertext bit is a function of every plaintext bit and every key bit (a thoroughly random-looking function). So 8 rounds *seem* enough.
- **But** experiments showed DES versions with fewer than 16 rounds are **more vulnerable to known-plaintext attacks than to brute force**. This justifies the designers' choice of 16.

Known-plaintext attack: the attacker has some plaintext–ciphertext pairs and tries to recover the key.

> **Remember:** 16 rounds, not 8. Extra rounds give a security margin against attacks on reduced-round versions.

Jayasham ND
4MC23CS059

K Aprameya Hebbar
4MC23CS063

Multiple DES

Data Encryption Standard (DES) | Chapter 6, Section 6.4 | Double DES, meet-in-the-middle attack and Triple DES

1. Overview: Why Multiple DES?

The major criticism of the original Data Encryption Standard (DES) is its 56-bit effective key length. DES processes 64-bit blocks, but its relatively short key makes exhaustive key search feasible with sufficiently powerful hardware and parallel processing. The aim of Multiple DES is to improve security while continuing to use the existing DES algorithm.

Approach	Description	Practical implication
Replace DES	Design and deploy a stronger new cipher, such as AES (discussed in Chapter 7).	Requires a transition to different software or hardware.
Cascade DES	Apply DES two or three times, using multiple keys.	Reuses existing DES implementations.

Multiple DES follows the second approach. Cascading means that the output of one encryption stage becomes the input of the next. However, simply doubling the number of keys does not automatically double the security: attacks on the composition must also be considered.

2. Mathematical Foundation: Composition and the Group Question

A mapping is a transformation from an input block to an output block. DES, for each fixed key, acts as a permutation on the set of 64-bit blocks. There are 2^64 possible block values. If the family of DES transformations formed a group under composition, two consecutive DES operations would be equivalent to one DES operation with some third key.

E_k2(E_k1(P)) = E_k3(P) (hypothetical closure)

The closure property would make Double DES pointless: the cascade would merely behave like another single-key DES. The textbook emphasizes that DES is not a group, so this collapse cannot generally be assumed.

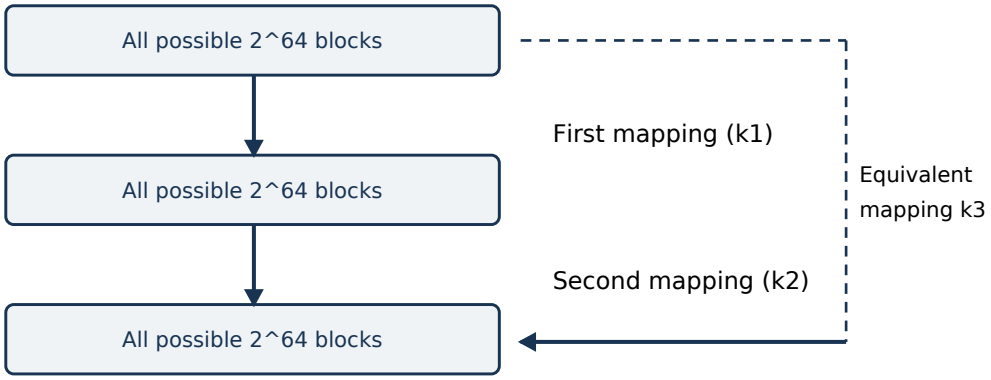


Figure 6.13 (redrawn): Composition of mappings. Two consecutive transformations could collapse to a third if the family were closed under composition.

2.1 First argument: The number of possible mappings

With N = 2^64 possible 64-bit inputs, the number of possible one-to-one mappings (permutations) is N! = (2^64)!, an astronomically large number. To index every possible permutation would require approximately log2((2^64)!) ≈ 2^70 bits of key information. DES uses only 56 effective key bits, so it represents a tiny fraction of all possible permutations.

This shows that DES cannot be the group of all permutations of 64-bit blocks. By itself, that observation does not exclude a smaller subgroup.

2.2 Second argument: Could DES form a smaller group?

The other possibility is that DES transformations form a subgroup of the full permutation group. The textbook states that mathematical results rule out the relevant subgroup possibility for the 56-bit DES family. Consequently, the DES family is not closed under composition, and a single DES key k3 generally cannot reproduce an arbitrary pair of DES encryptions.

In general: E_k2(E_k1(P)) ≠ E_k3(P)

Conclusion: Multiple DES can genuinely produce transformations beyond a single DES operation. Whether those transformations are secure must be assessed separately.

3. Double DES (2DES)

Double DES applies the DES encryption algorithm twice with two independent 56-bit keys, k1 and k2. The plaintext first passes through DES with k1, producing an intermediate 64-bit block M. That intermediate block is encrypted with k2 to produce the final ciphertext C. Decryption reverses the stage order and uses the corresponding DES inverse operations.

Encryption: C = E_k2(E_k1(P))

Decryption: P = D_k1(D_k2(C))

The nominal combined key length is 112 bits (56 + 56). At first sight, exhaustive search over all key pairs appears to require 2^112 trials. This intuition overlooks a much more efficient known-plaintext attack.

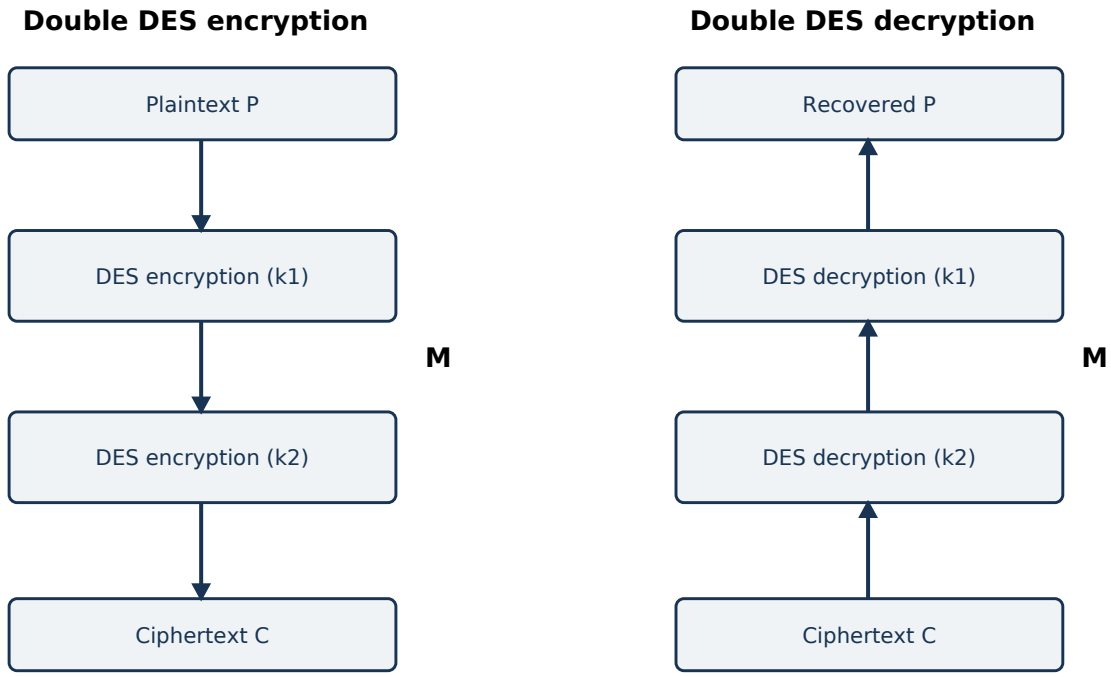


Figure 6.14 (redrawn): Double DES encryption and reverse decryption. Both paths meet at the same 64-bit intermediate value M.

Property	Single DES	Double DES
Number of DES stages	1	2
Effective key material	56 bits	112 bits
Naive key-pair search	2^56	2^112
Important attack	Exhaustive key search	Meet-in-the-middle

4. Meet-in-the-Middle (MITM) Attack

A known-plaintext attack assumes the attacker possesses at least one matching plaintext-ciphertext pair (P, C). For Double DES, the intermediate value M must be the same whether calculated forward from P or backward from C:

M = E_k1(P) and M = D_k2(C)

This shared intermediate value enables the attacker to attack the two DES stages separately rather than trying all 2^112 combinations. In the textbook example, Eve intercepts a plaintext-ciphertext pair produced by Alice and uses it to search for the two keys.

4.1 Step-by-step attack procedure

Step 1: Forward computation. Eve encrypts the known plaintext P under every possible 56-bit value of k1. She stores the resulting 2^56 intermediate values M alongside the corresponding k1 values.

Step 2: Backward computation. Eve decrypts the known ciphertext C under every possible 56-bit value of k2. She stores the resulting 2^56 intermediate values M alongside the corresponding k2 values.

Step 3: Meet in the middle. Eve sorts, hashes, or otherwise indexes the intermediate values and searches for equal M values across the two collections. Every match yields a candidate key pair (k1, k2). At least the actual key pair must be among the candidates.

Step 4: Eliminate false matches. If multiple candidate pairs remain, Eve tests them against another known plaintext-ciphertext pair. She repeats this verification with further intercepted pairs until a unique key pair remains.

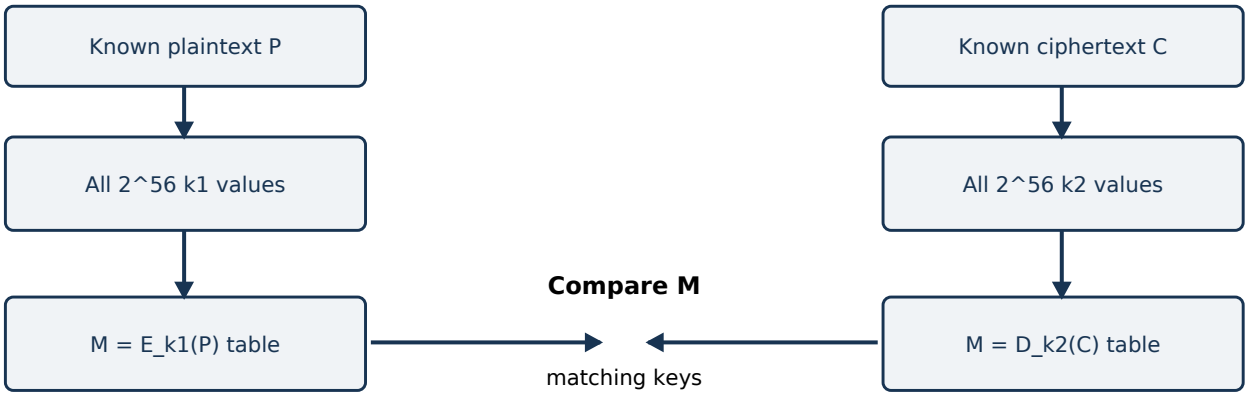


Figure 6.15 (redrawn): Two lookup tables are matched on the common 64-bit intermediate text M.

4.2 Why the attack is much faster

The dominant computation is approximately 2^56 DES encryptions plus 2^56 DES decryptions, giving roughly 2^57 DES operations, plus lookup and candidate-verification overhead. The attack also requires substantial storage, typically on the order of 2^56 table entries in the straightforward form.

Work ≈ 2^56 + 2^56 = 2^57 DES operations

Because the intermediate block has only 64 bits, accidental matches can occur between unrelated keys. Therefore, matching a single intermediate value does not always identify the correct pair; the extra known pairs are essential for filtering candidates.

Attack aspect	Explanation
Attacker knowledge	One or more known plaintext-ciphertext pairs.
Forward table	E_k1(P) for all 2^56 first-stage keys.
Backward table	D_k2(C) for all 2^56 second-stage keys.
Match condition	E_k1(P) = D_k2(C).
Main computational cost	About 2^57 DES operations, not 2^112.
Additional requirement	Very large memory and verification of possible false matches.

Key takeaway: Moving from single DES to Double DES does not increase resistance to this attack from 2^56 to 2^112. Under the textbook attack model, the increase in computational work is only to about 2^57, making Double DES an unattractive security improvement.

5. Triple DES (3DES)

Triple DES was proposed to overcome the weakness of Double DES by applying three DES stages. The textbook describes two versions: two-key Triple DES and three-key Triple DES. Both use a 64-bit block and preserve compatibility with the original DES through an Encrypt–Decrypt–Encrypt (EDE) arrangement.

5.1 Triple DES with two keys (2-key 3DES)

Two-key Triple DES uses two distinct 56-bit keys, k1 and k2. The first and third stages use k1, while the middle stage uses k2. Importantly, the middle stage is DES decryption during the overall encryption process, producing the EDE sequence.

Encryption: C = E_k1(D_k2(E_k1(P)))

Decryption: P = D_k1(E_k2(D_k1(C)))

The combined key material is 112 bits. Using EDE instead of three consecutive encryptions provides backward compatibility: if k1 = k2 = k, the middle decryption cancels one of the encryptions, and the result becomes ordinary DES encryption under k.

If k1 = k2 = k: E_k(D_k(E_k(P))) = E_k(P)

The textbook notes that two-key 3DES remains vulnerable to known-plaintext attacks, but it is substantially stronger than Double DES. Historically, this form was adopted in banking applications.

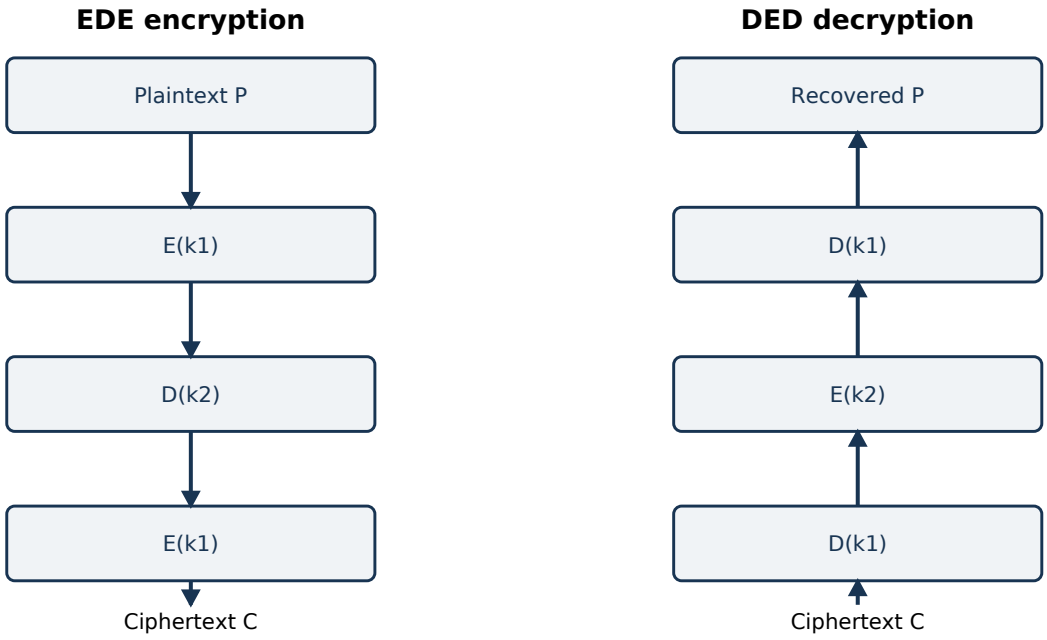


Figure 6.16 (redrawn): Two-key Triple DES uses E–D–E for encryption and D–E–D for decryption, with k1 reused in the outer stages.

5.2 Triple DES with three keys (3-key 3DES)

Three-key Triple DES uses independent keys k1, k2, and k3, each with 56 effective bits, for a nominal total of 168 bits of key material. It retains the same EDE arrangement for DES compatibility, but the third stage now uses k3 rather than repeating k1.

Encryption: C = E_k3(D_k2(E_k1(P)))

Decryption: P = D_k1(E_k2(D_k3(C)))

For compatibility with single DES, setting k1 = k2 = k3 = k makes the EDE sequence reduce to E_k(P). More generally, setting k1 = k and k2 = k3 to the same arbitrary key also reduces the operation to E_k(P), since the adjacent D_k2 and E_k3 cancel.

The textbook mentions applications such as PGP as users of three-key Triple DES. The additional independent key is intended to improve security compared with the two-key version.

5.3 Two-key and three-key Triple DES compared

Feature	Two-key 3DES	Three-key 3DES
Key arrangement	k1, k2, k1	k1, k2, k3
Nominal key material	112 bits	168 bits
Encryption stages	E(k1) → D(k2) → E(k1)	E(k1) → D(k2) → E(k3)
Decryption stages	D(k1) → E(k2) → D(k1)	D(k3) → E(k2) → D(k1)
Single DES compatibility	Yes, set k1 = k2	Yes, set all keys equal
Textbook application note	Banking industry	PGP and other applications

6. Consolidated Comparison and Revision

Cipher	DES operations	Nominal key bits	Main security observation
DES	1	56	Short key vulnerable to brute-force search.
Double DES	2	112	MITM reduces work to roughly 2^57 operations.
2-key Triple DES	3 (EDE)	112	Much stronger than 2DES, though attacks exist.
3-key Triple DES	3 (EDE)	168	Three independent keys; stronger than two-key variant.

6.1 Frequently confused points

Key length versus security strength: The total number of key bits does not by itself equal the effective attack complexity. Double DES has 112 bits of key material but is vulnerable to an attack requiring approximately 2^57 operations.

Encryption versus decryption order: For a cascade, decryption applies inverse operations in reverse order. The last encryption stage must be undone first.

Why EDE instead of EEE? The middle DES decryption stage enables backward compatibility with single DES when the appropriate keys are equal.

Group versus non-group: Showing that DES is not the full permutation group is not by itself a complete group proof; the subgroup argument is also needed.

6.2 Short worked conceptual example

Suppose Alice encrypts plaintext P using Double DES with secret keys k1 and k2. She computes M = E_k1(P) and then C = E_k2(M). Eve knows P and C but not either key. Instead of guessing every (k1, k2) pair, Eve computes all possible values of E_k1(P) and D_k2(C). When the two lists share a value M, the associated keys form a candidate. A second known pair distinguishes genuine keys from accidental collisions. This is the essential idea behind the meet-in-the-middle attack.

6.3 Exam-ready recap

DES uses a 56-bit effective key and a 64-bit block. Cascading DES was proposed as a way to reuse the existing cipher while increasing key material. DES is not closed under composition, so repeated encryption does not generally collapse into a single DES transformation. Double DES uses two independent keys but is vulnerable to the meet-in-the-middle known-plaintext attack. Triple DES uses three stages; its two-key and three-key variants employ EDE encryption and DED decryption for backward compatibility.

Context note: The descriptions of historical adoption reflect the supplied textbook section. For modern new systems, Triple DES is considered legacy technology; AES is the preferred modern block cipher.

Reference

Behrouz A. Forouzan, Introduction to Cryptography and Network Security, Chapter 6, Section 6.4, “Multiple DES,” textbook figures 6.13–6.16. Diagrams in these notes have been independently redrawn for clarity.